



哈爾濱工業大學
HARBIN INSTITUTE OF TECHNOLOGY



分布式机器学习(II)

模型并行

王宏志

哈尔滨工业大学

计算学部 海量数据计算研究中心

wangzh@hit.edu.cn

<http://homepage.hit.edu.cn/wang>

目录

Contents

- 1 张量并行
- 2 上下文并行
- 3 流水线并行
- 4 专家并行
- 5 并行对比与扩展

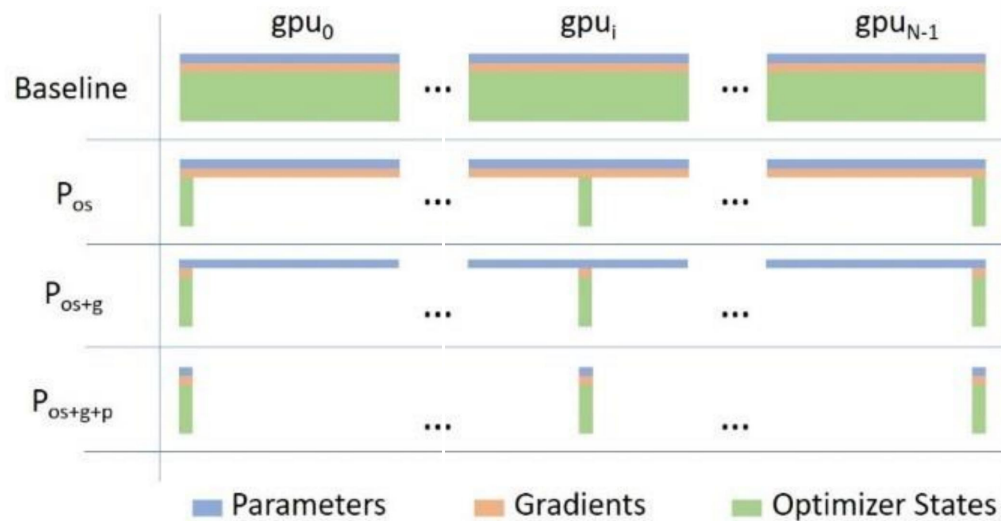
目录

Contents

- 1 张量并行
- 2 上下文并行
- 3 流水线并行
- 4 专家并行
- 5 并行对比与扩展

ZeRO-DP: ZeRO 支持的数据并行性

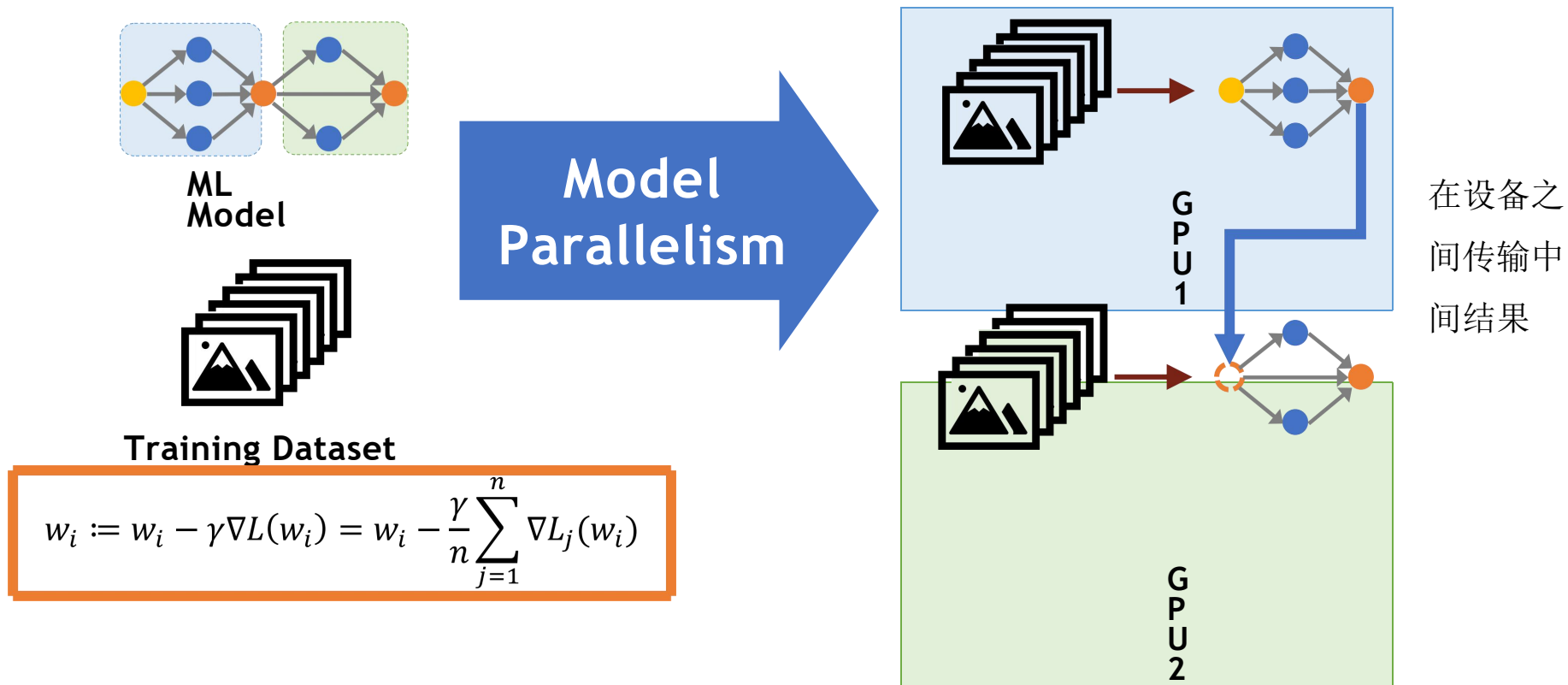
- ZeRO消除跨数据并行进程的冗余
- Stage 1:分区优化器状态
- Stage 2:分区梯度
- Stage 3:分区参数



然而，这里有一些限制： DP仅在模型的一层适合单个GPU时才工作，ZeRO只能划分参数、梯度和优化器状态，而不能划分激活内存！回想一下关于激活内存的讨论，这部分内存的规模与序列长度和批量大小成正比。本可以限制这些，但在实践中，不想因为硬件限制而只能使用短序列长度进行训练。

模型并行

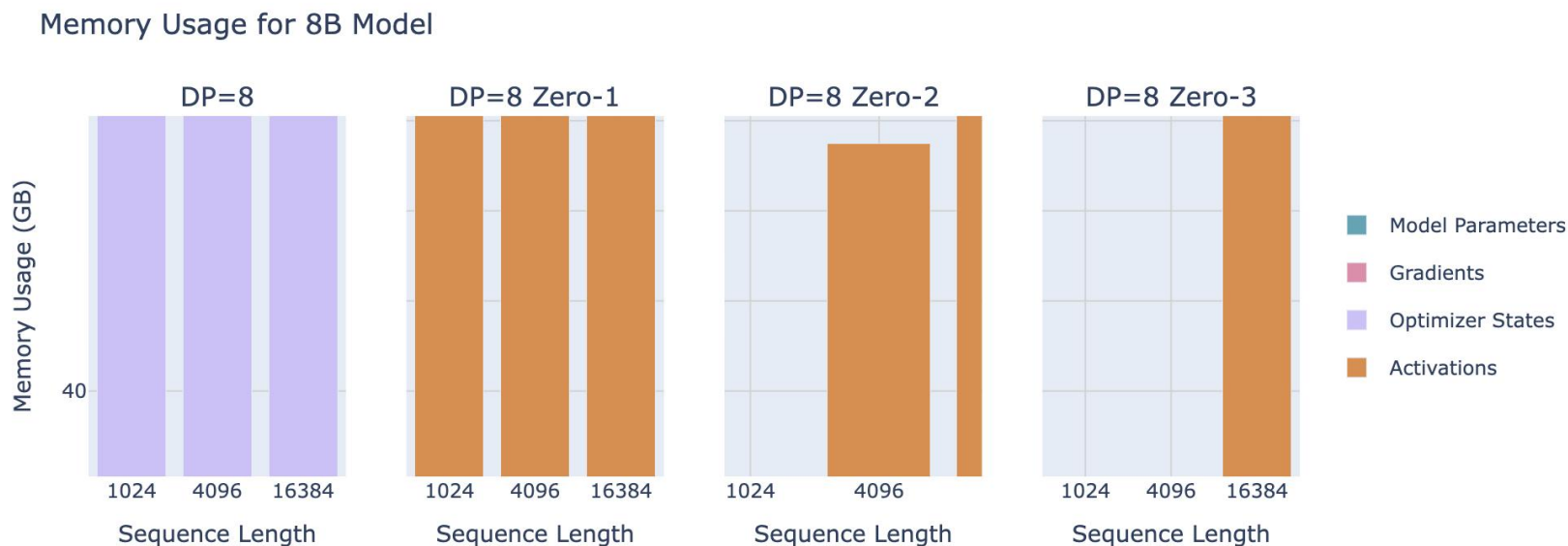
- 将模型拆分为多个子图并将它们分配到不同的设备上



ZeRO-DP: ZeRO 支持的数据并行性

为了克服这个问题

是时候考察一个新的、正交的并行轴——张量并行（TP）。与依赖于大量参数通信的ZeRO-3不同，TP提议将参数、梯度、优化器状态以及激活在设备间进行分片，而不需要在GPU之间进行任何模型参数的通信。



张量并行

张量并行

张量并行（TP），一种将权重、梯度和优化器状态以及激活都进行分片的方法——而且无需在计算之前将它们全部收集。让首先看看TP如何与简单的矩阵乘法（matmul）操作一起工作。

✓ 张量并行利用矩阵乘法（ $A \times B$ ）的数学性质。

为了理解其工作原理，让考察两个使这种并行化成为可能的基本方程：

$$1. A \cdot B = A \cdot [B_1 \quad B_2 \quad \cdots] = [AB_1 \quad AB_2 \quad \cdots]$$

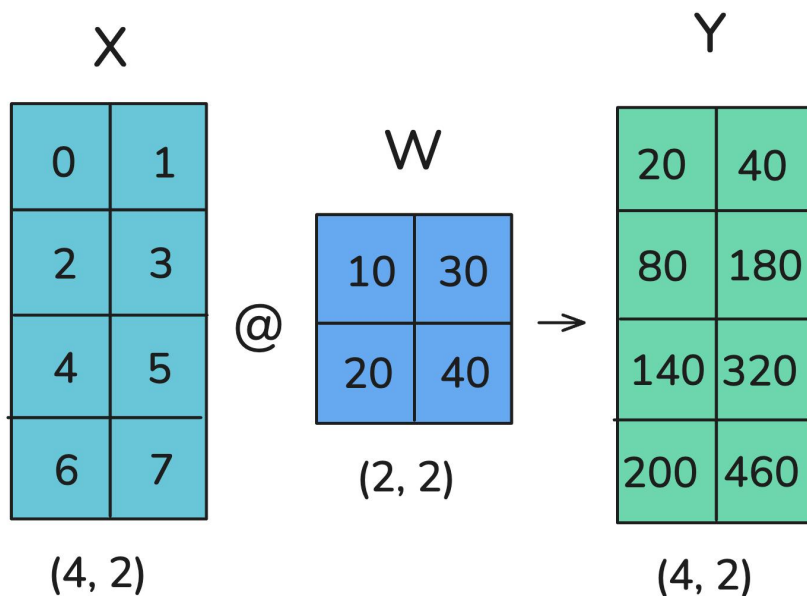
$$2. A \cdot B = [A_1 \quad A_2 \quad \cdots] \begin{bmatrix} B_1 \\ B_2 \\ \vdots \end{bmatrix} = \sum_{i=1}^n A_i B_i$$

这意味着可以通过单独乘以矩阵 B 的每一列或者单独乘以每一行并将结果组合起来来计算矩阵乘积。在神经网络中，矩阵乘积通常以格式 $X \times W$ 表示，其中： X 代表输入或激活值， W 代表线性层的权重

张量并行

如何并行化

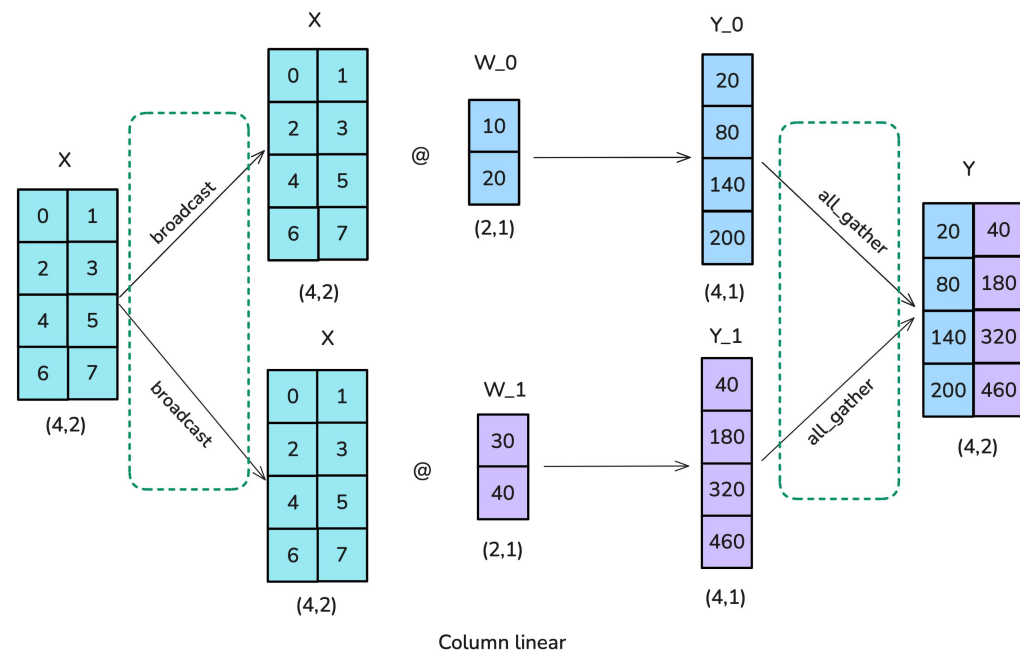
在张量并行中，张量沿着特定维度分成N个碎片，并分布在N个GPU上。矩阵可以在列或行上分割，导致行或列并行。正如将在下面的讨论中看到的那样，行和列分割需要不同的通信原语。



张量并行

如何并行化

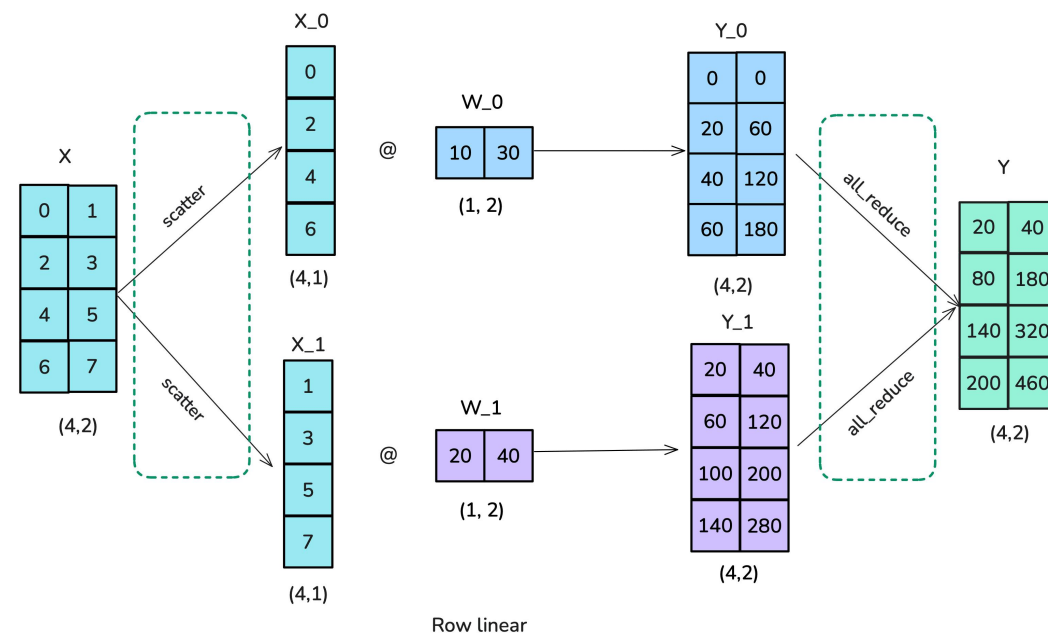
的第一个选项是使用列式（也称为列线性）分片：将完整输入矩阵复制到每个工作节点，需要执行一个称为广播的操作，并按列分割权重矩阵。然后，输入与部分权重矩阵相乘，最后使用全收集操作合并结果。



张量并行

如何并行化

第二种选项被称为行式（或行线性）分片。行线性意味着将权重矩阵分成行块。然而，这也要求分割输入，因此需要使用`scatter`操作而不是列线性分片中使用的广播操作。每个工作者的结果已经处于正确的形状，但要求和以得到最终结果，因此这种场景也要求使用All-Gather操作



数据和张量模型并行的通讯量

- 数据并行性: $O(C_{out} * C_{in})$

数据并行中, 每个GPU持有完整模型副本, 处理不同数据子集。通信主要发生在梯度同步阶段, 需要将所有GPU计算的梯度进行All-Gather操作。

- 张量模型并行性(分区输出): $O(B * C_{in})$

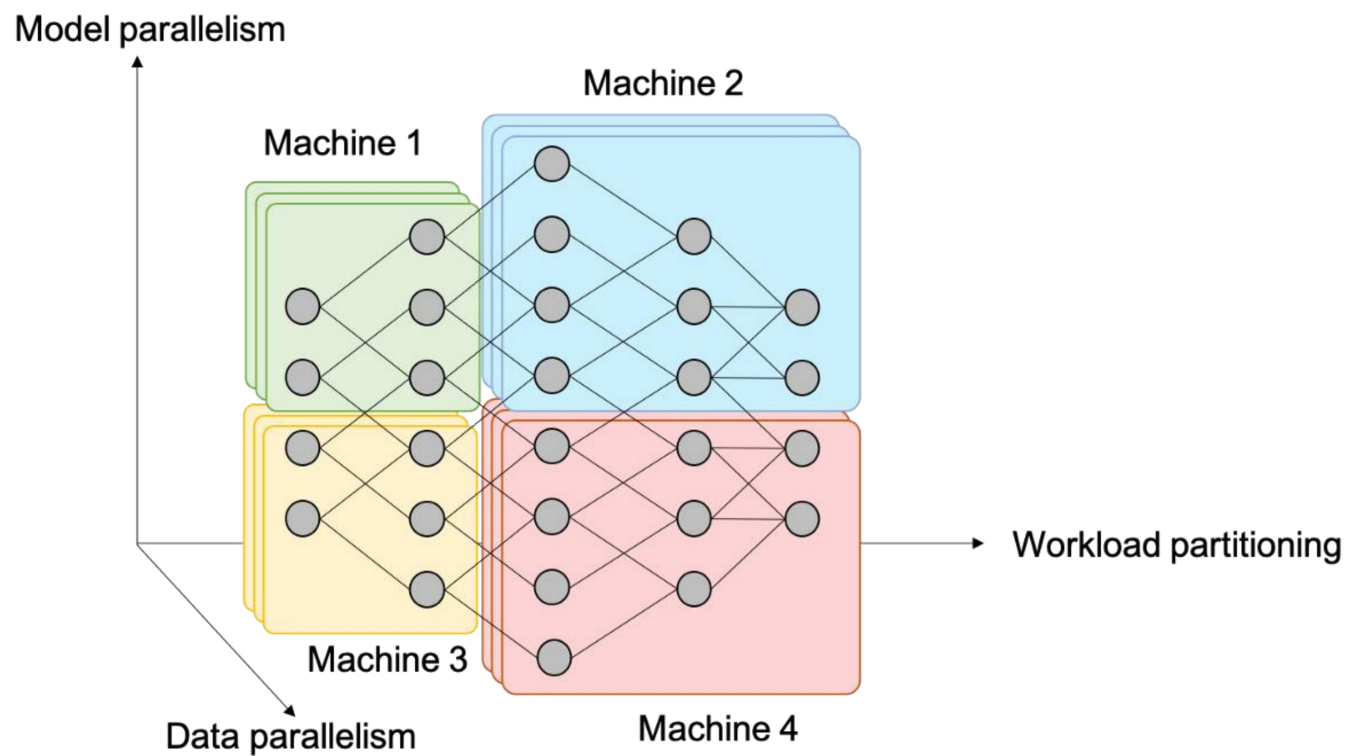
在张量并行中, 模型的权重矩阵(如全连接层)按列切分到不同GPU上。"分区输出"指每个GPU只计算部分输出通道。

- 张量模型并行性(减少输出): $O(B * C_{out})$

与"分区输出"相反, "减少输出"指将权重矩阵按行切分, 每个GPU计算完整输出但只处理部分输入

- 最佳策略取决于模型规模和硬件资源

结合数据和模型并行性



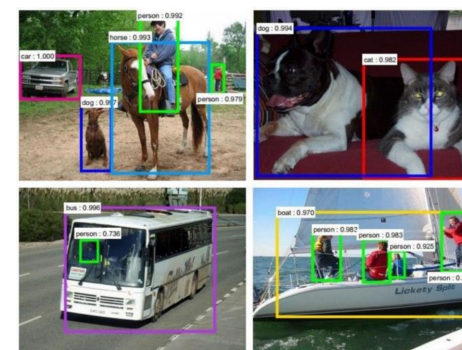
示例:卷积神经网络



分类



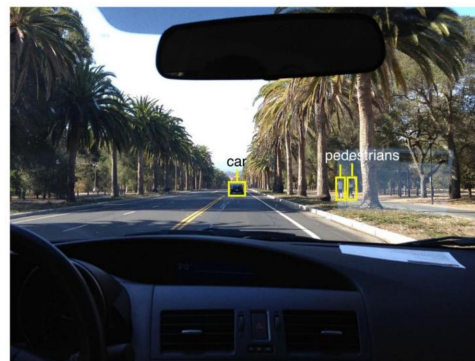
检索



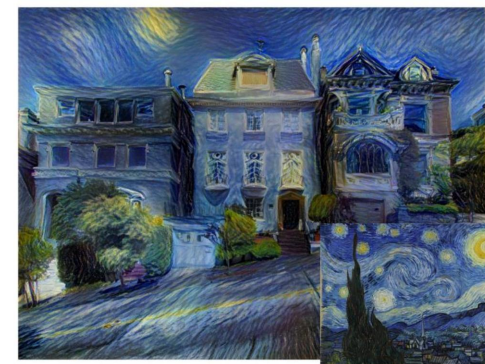
检测



细分



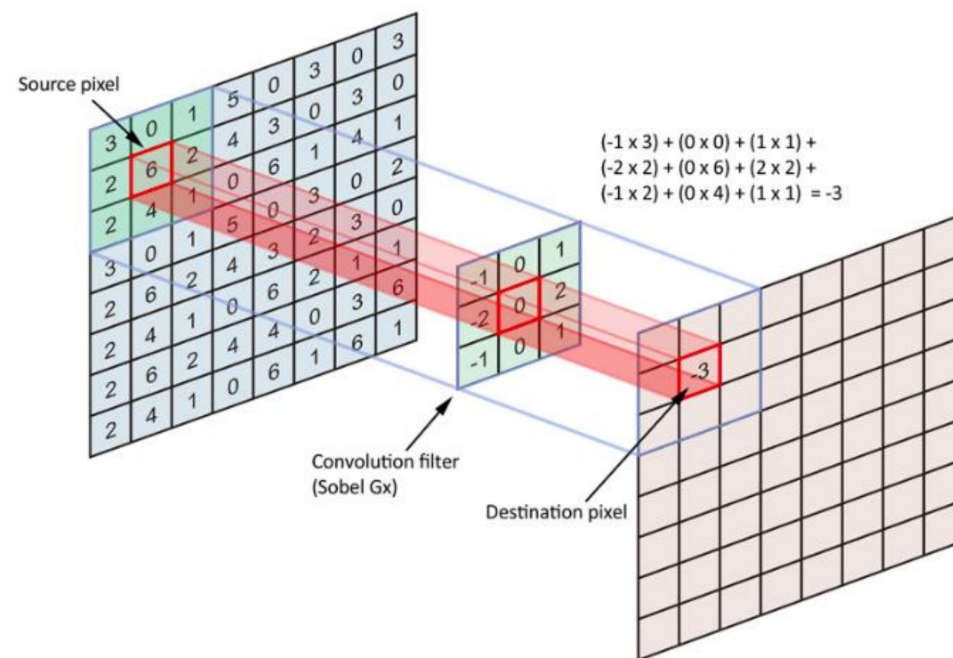
自动驾驶



综合

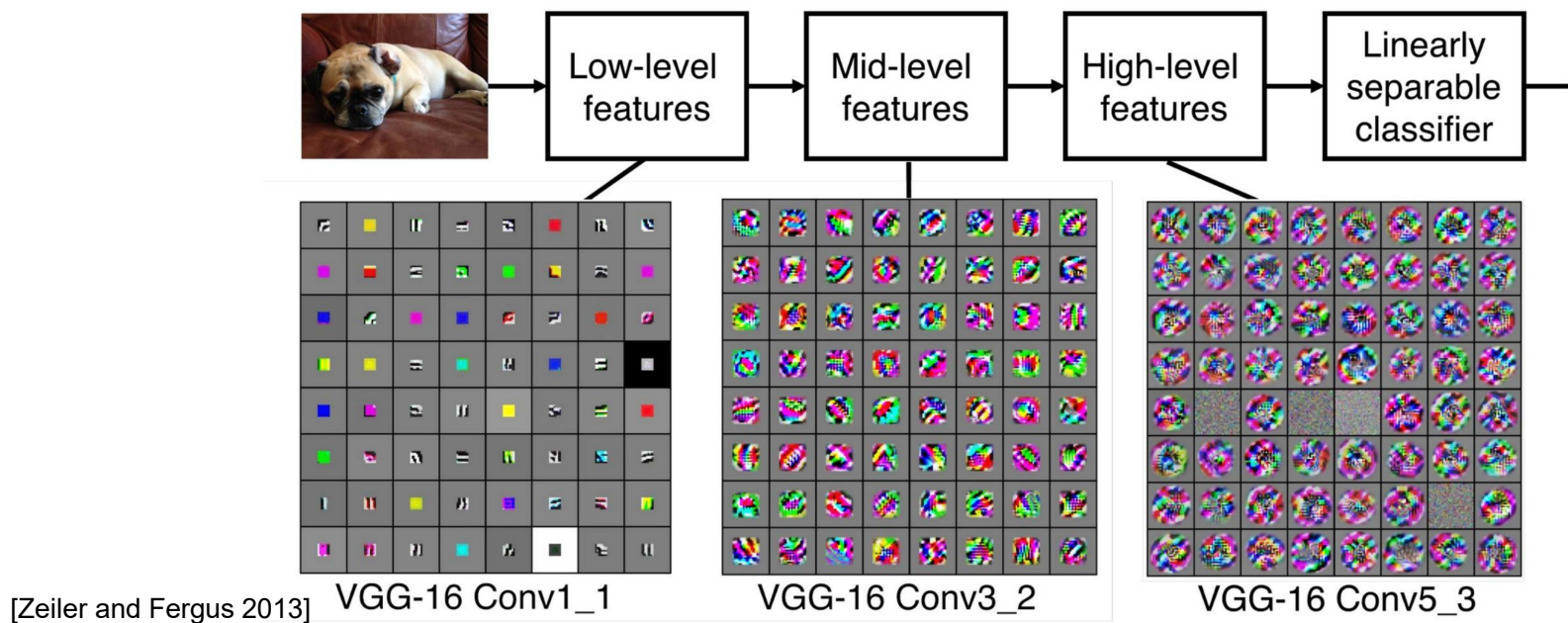
卷积

- 将滤镜与图像卷积: 在空间上滑过图像, 然后计算点积



CNNs

- 卷积层的序列, 中间有池化、归一化、和激活函数



并行卷积神经网络

- 卷积层
- 计算的 90-95%
- 参数的 5%
- 非常大的中间激活

数据并行性

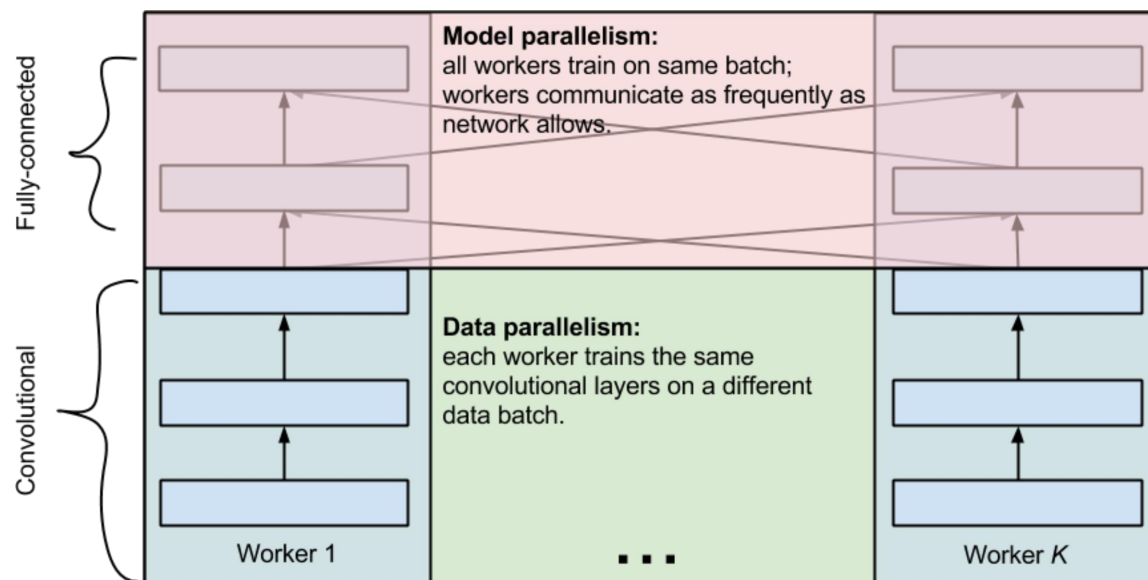
- 全连接层
- 计算的 5-10%
- 参数的 95%
- 小的中间激活

张量模型并行性

- 讨论: 如何并行化 CNN?

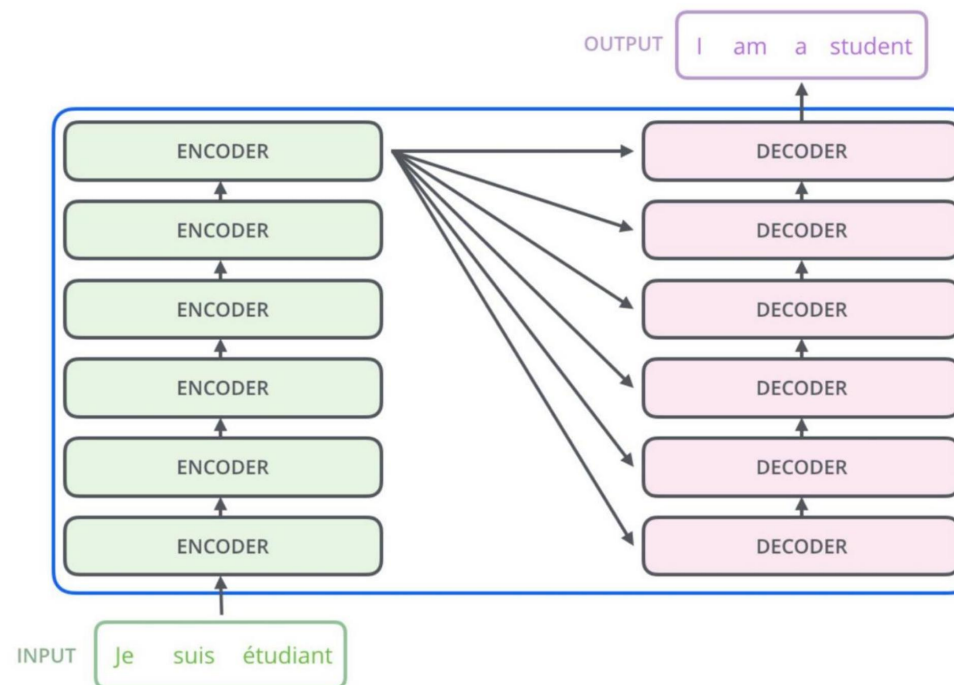
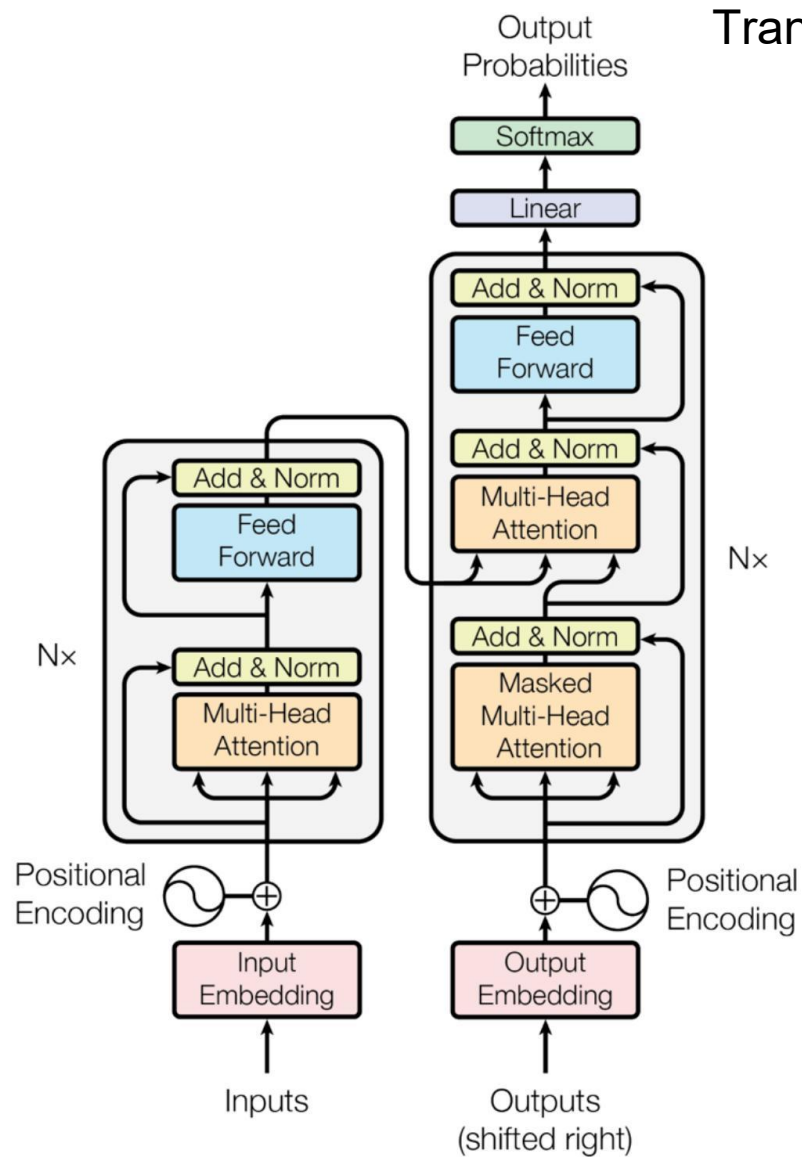
并行卷积神经网络

- 卷积层的数据并行性
- 全连通层的张量模型并行性

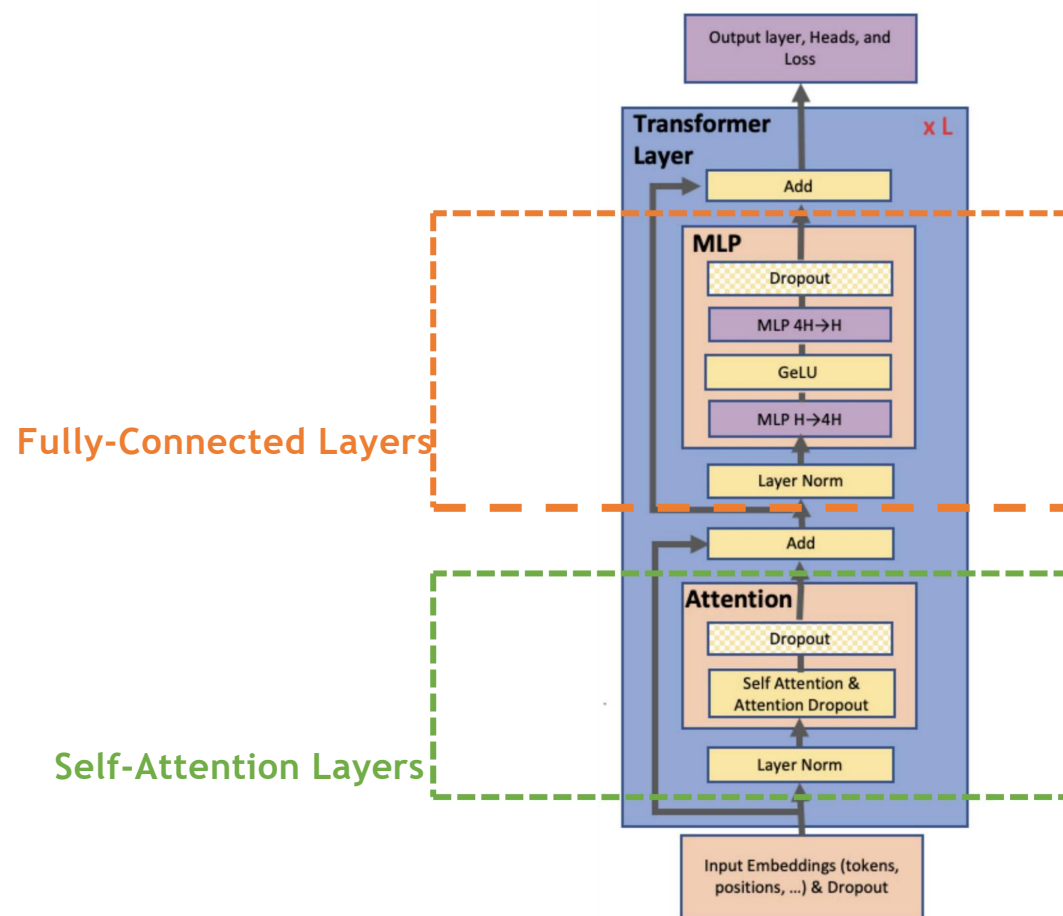


示例：并行化Transformer

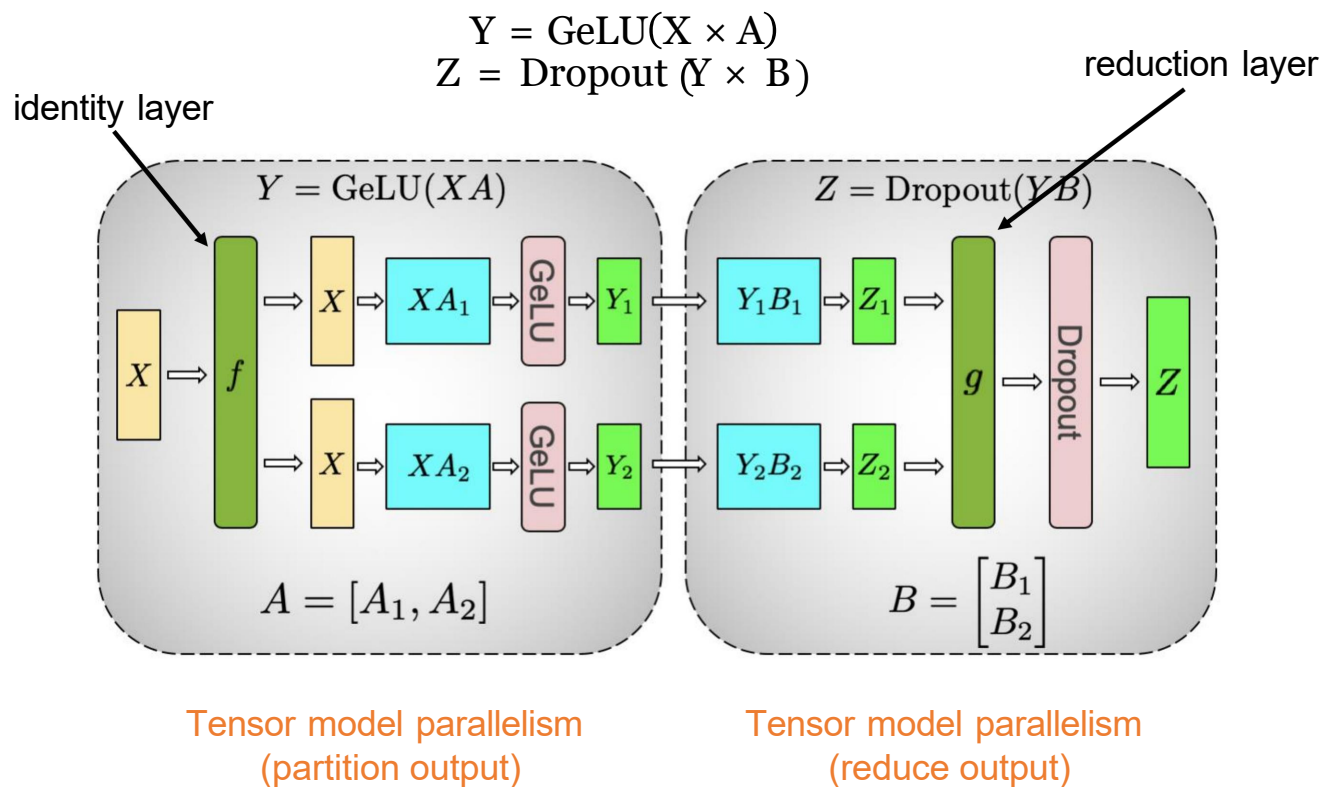
Transformer: 语言理解中的注意力机制



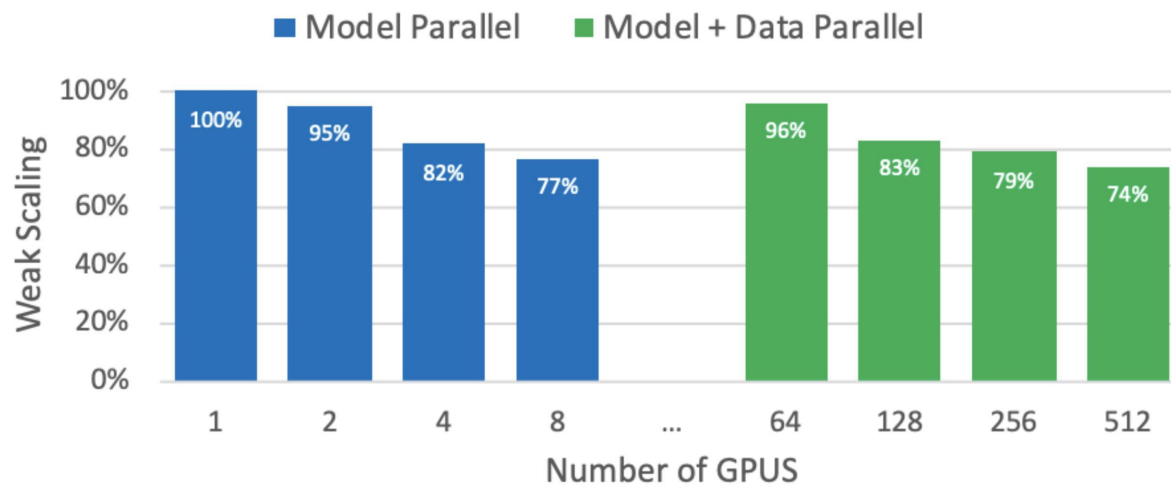
单 Transformer Layer



Transformer中的全连接层并行化



并行化Transformers

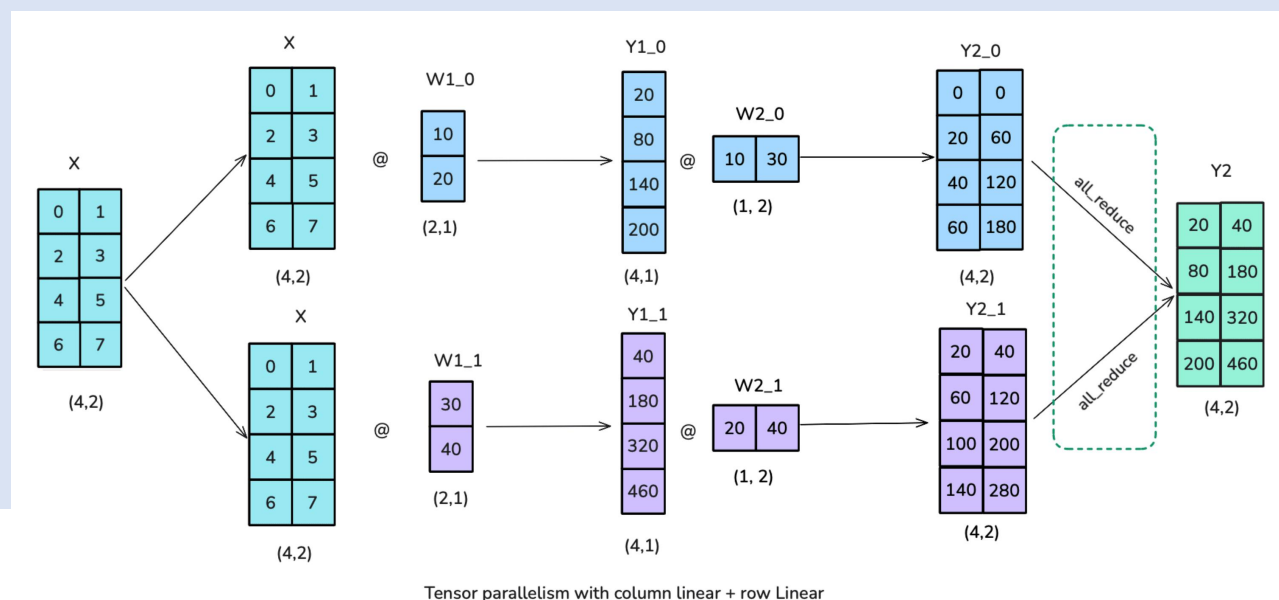


通过结合数据和模型并行性扩展到 512 个 GPU

Transformer块中的张量并行

Transformer模型

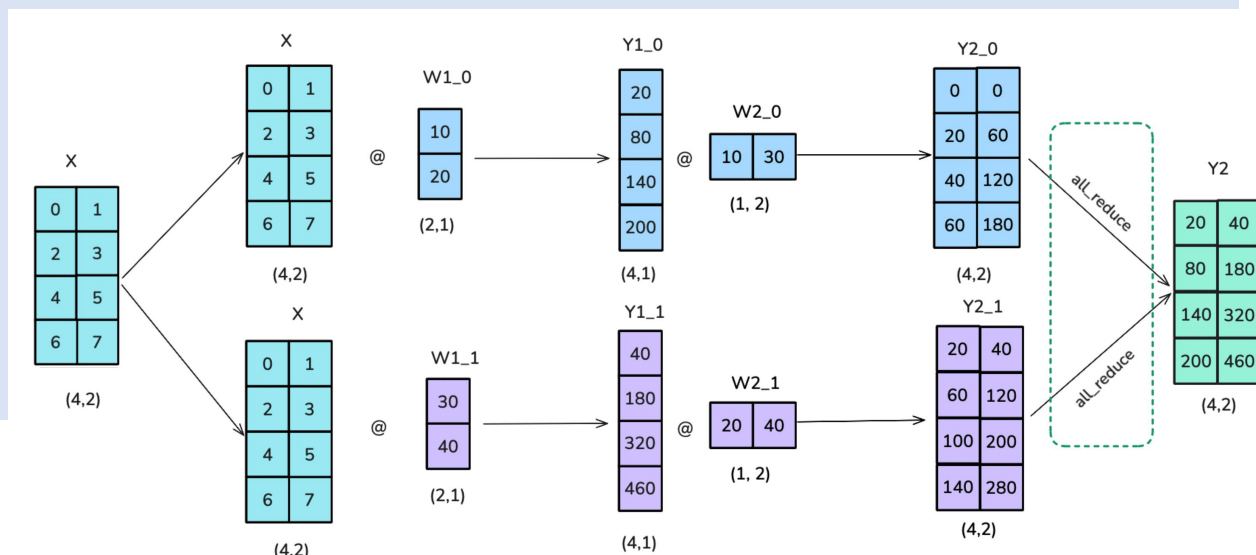
Transformer模型由两个主要构建块组成：一个前馈多层感知器（MLP）块和一个多头注意力（MHA）块。可以将张量并行应用于两者。



Transformer块中的张量并行

Transformer模型

前馈部分可以通过先进行列线性分割，然后进行行线性分割来实现并行化，这相当于广播以复制输入，并在前向传递中进行All-Gather。请注意，在实际训练中不需要广播，因为已经确保输入在TP排名之间已经同步。这种设置比先进行行线性分割然后进行列线性分割更高效，因为可以在分割操作之间跳过中间的All-Gather。

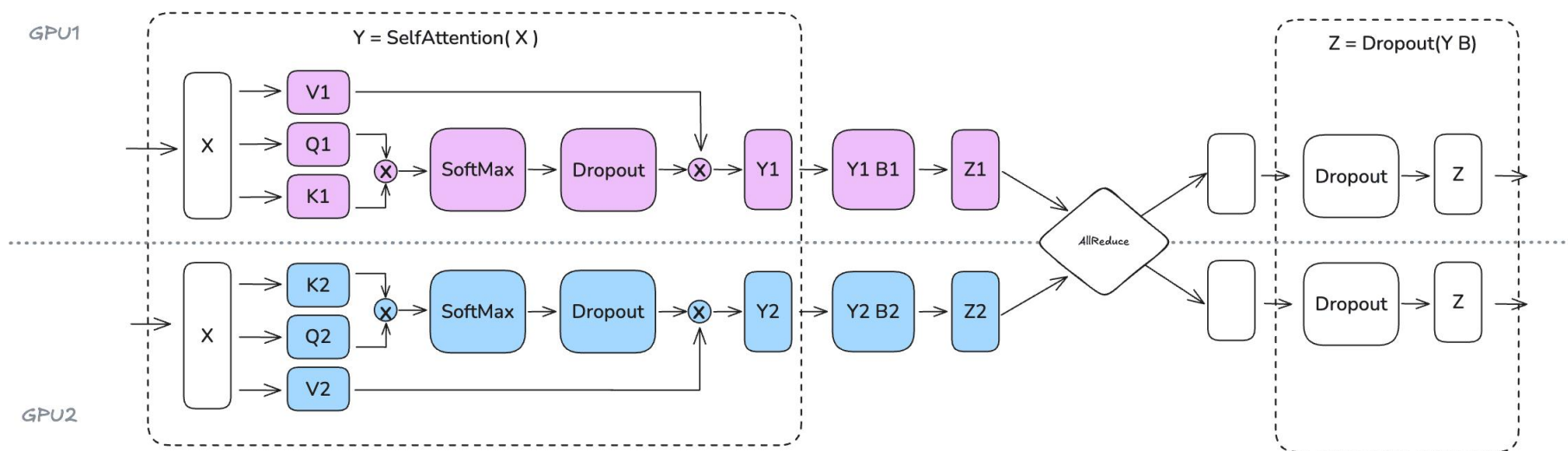


Tensor parallelism with column linear + row Linear

Transformer块中的张量并行

多头注意力模块

可以一般地采用类似的方法，其中查询（Q）、键（K）和值（V）矩阵以列并行方式分割，输出投影可以被认为是行线性的。在多头注意力中，列并行方法有一个非常自然的解释：每个GPU计算单个或注意力头子集的注意力。同样的方法也适用于多查询注意力（MQA）或分组查询注意力（GQA），其中键和值在查询之间共享。



Transformer块中的张量并行

有效地将张量并行应用于注意力和MLP块

因为它们具有自然独立的维度。注意力块可以沿着`num_attention_heads`维度并行化，因为每个注意力头独立操作。同样，MLP块可以沿着`hidden_dim`维度并行化，因为前馈网络中的操作在这个维度上是独立的。

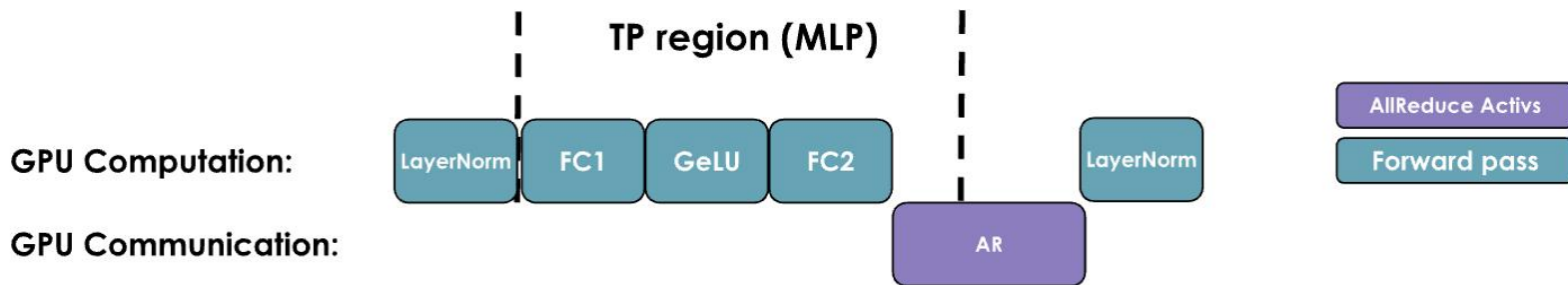
✓ 值得注意的是，张量并行度不应超过注意力头的数量

- 因为沿着`num_attention_heads`维度对QKV投影进行了分片。在使用分组查询注意力（GQA）时，有 `num_attention_heads` 查询头，但只有 `num_kv_heads` 键 / 值头（ $num_attention_heads \geq num_kv_heads$ ）。在这种情况下，仍然可以将 TP 设置为 `num_attention_heads`，但需要确保 K/V 头在 GPU 之间保持适当的同步。
- 例如，Llama-3 8B 有 32 个查询头，但只有 8 个键/值头，所以虽然理论上的 TP 度数可以达到 32，但仍需要谨慎实现以保持张量并行worker之间的 K/V 头同步。

Transformer块中的张量并行

张量并行并不是训练的万能药

在模型的计算路径中直接添加了几个分布式通信原语，因此很难完全隐藏/重叠与计算（就像在ZeRO中做的那样），所以最终的最终性能将是计算和内存增益与增加的通信开销之间的权衡结果。



观察张量并行MLP（MHA同理）的操作时间线，可以更好地理解其中涉及的权衡。在每个解码器层的正向传递中，会遇到一个与All-Gather操作同步的点，这个点不能与计算重叠。这种暴露的通信开销是必要的，以便在应用最终的LayerNorm之前，将张量并行秩的局部结果组合起来。

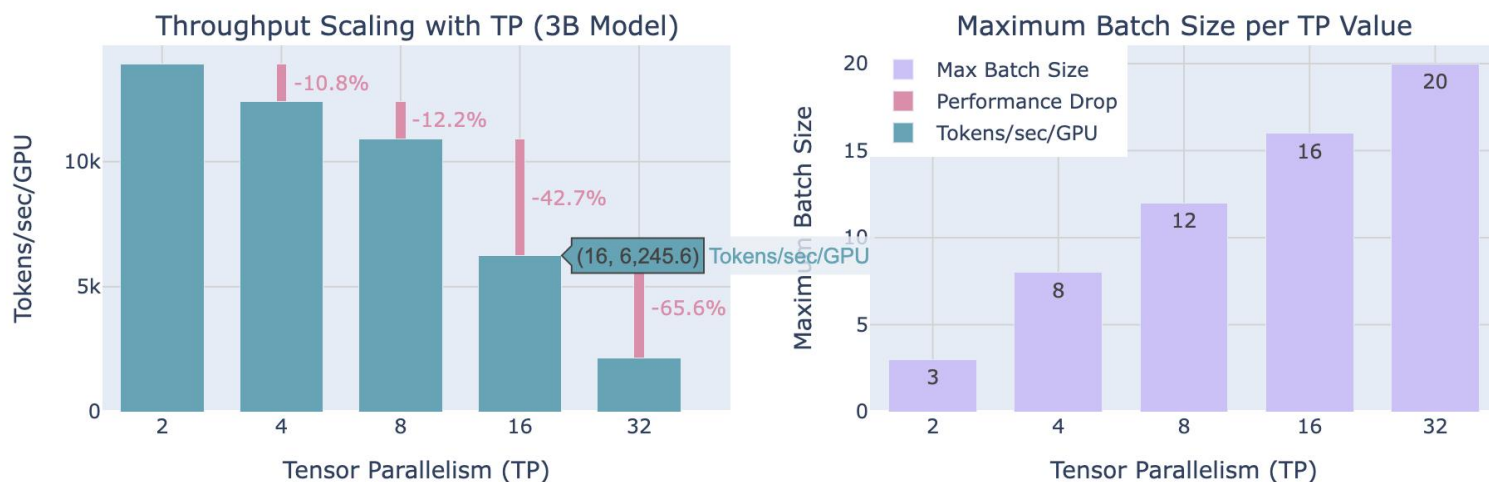
Transformer块中的张量并行

张量并行确实有助于减少矩阵乘法中的激活内存

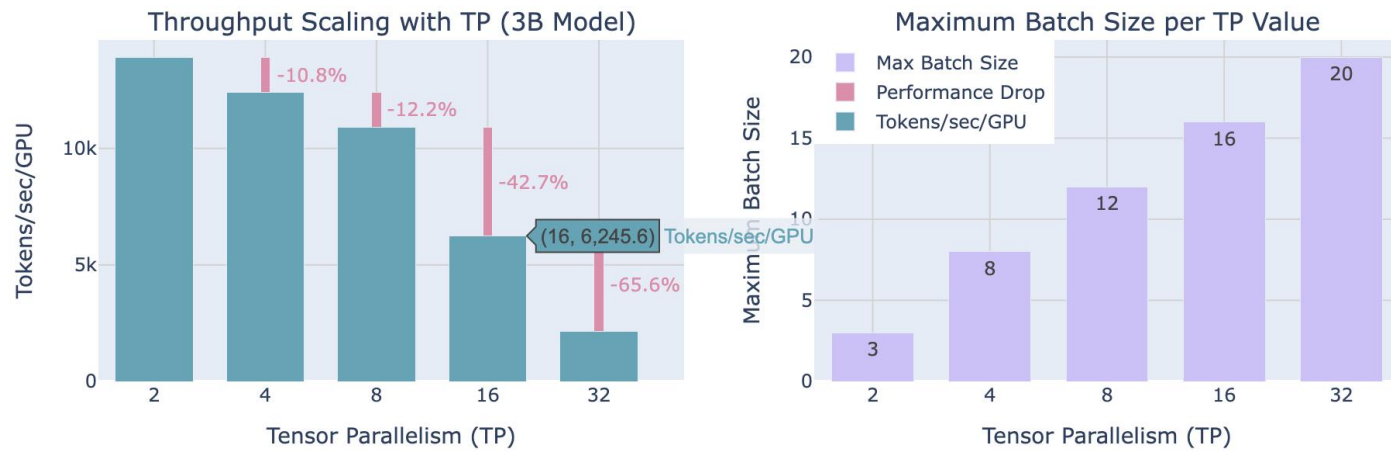
因为中间激活被分散到多个GPU上。然而，仍然需要收集完整的激活以进行如LayerNorm等操作，这意味着没有获得本可以得到的全部内存效益。

此外，TP引入了显著的通信需求，这严重依赖于网络基础设施。无法完全隐藏这种特定的全归一化操作背后的计算意味着它直接增加了正向传播的关键路径，其中关键路径指的是确定完成正向传递所需最短时间的操作序列。

让在扩大TP程度时更好
地看看这个权衡



Transformer块中的张量并行



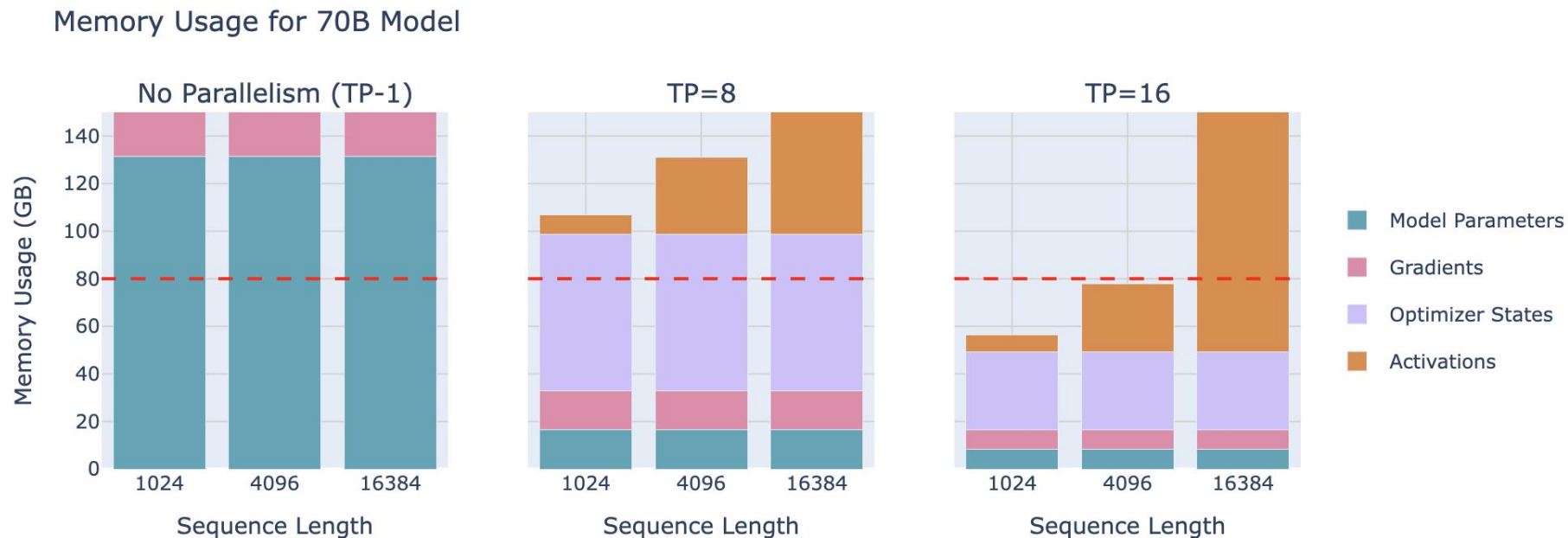
在扩大TP程度时更好观察这个权衡

- 在增加TP会导致每GPU吞吐量降低（左侧），但它使得处理更大的批量大小成为可能（右侧），这说明了分布式训练中计算效率与内存可用性之间的权衡。
- 在实践中，如图左侧所示，扩展到8个GPU以上时，张量并行通信开销变得更明显。虽然单个节点内张量并行可以利用快速的NVLink互连，但跨节点需要较慢的网络连接。从TP=8到TP=16时观察到显著的下降，从TP=16到TP=32时下降更为陡峭。在更高的并行度下通信开销变得过高以至于迅速主导计算时间。

Transformer块中的张量并行

说到这里

- 张量并行通过在多个GPU上分布模型参数、梯度、优化器状态和激活（在一定程度上）为内存使用提供了重要优势。让以一个70亿参数的模型为例来考察这种效果：



Transformer块中的张量并行

增加张量并行性

- 可以降低每个GPU上模型参数、梯度和优化器状态所需的内存，直到可以开始将更大的模型拟合到单个8GPU节点上。

有没有办法从这项技术中获得更多好处？

- 层归一化和dropout仍然需要在每个GPU上收集完整的激活，这在一定程度上抵消了内存节省。可以通过找到并行化这些剩余操作的方法来得做得更好。
- 关于张量并行训练中层归一化的一个有趣点是，由于每个TP排名在All-Gather之后看到相同的激活，层归一化的权重实际上不需要All-Gather来在反向传播后同步它们的梯度。它们自然地在排名之间保持同步。然而，对于dropout操作，必须确保在TP排名之间同步随机种子，以保持确定性行为。

序列并行

序列并行性 (SP)

序列并行性 (SP) 涉及将模型中由张量并行性未处理的部件的激活和计算进行拆分，例如dropout和LayerNorm，但沿着输入序列维度而不是隐藏维度进行拆分。

这是可行的

因为这些操作需要访问完整的隐藏维度以正确计算。例如，LayerNorm需要完整的隐藏维度来计算均值和方差：

$$\text{LayerNorm}(x) = \gamma \cdot \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta$$

其中 $\mu = \text{mean}(x)$ ， $\sigma^2 = \text{var}(x)$ 在隐藏维度 h 上计算。

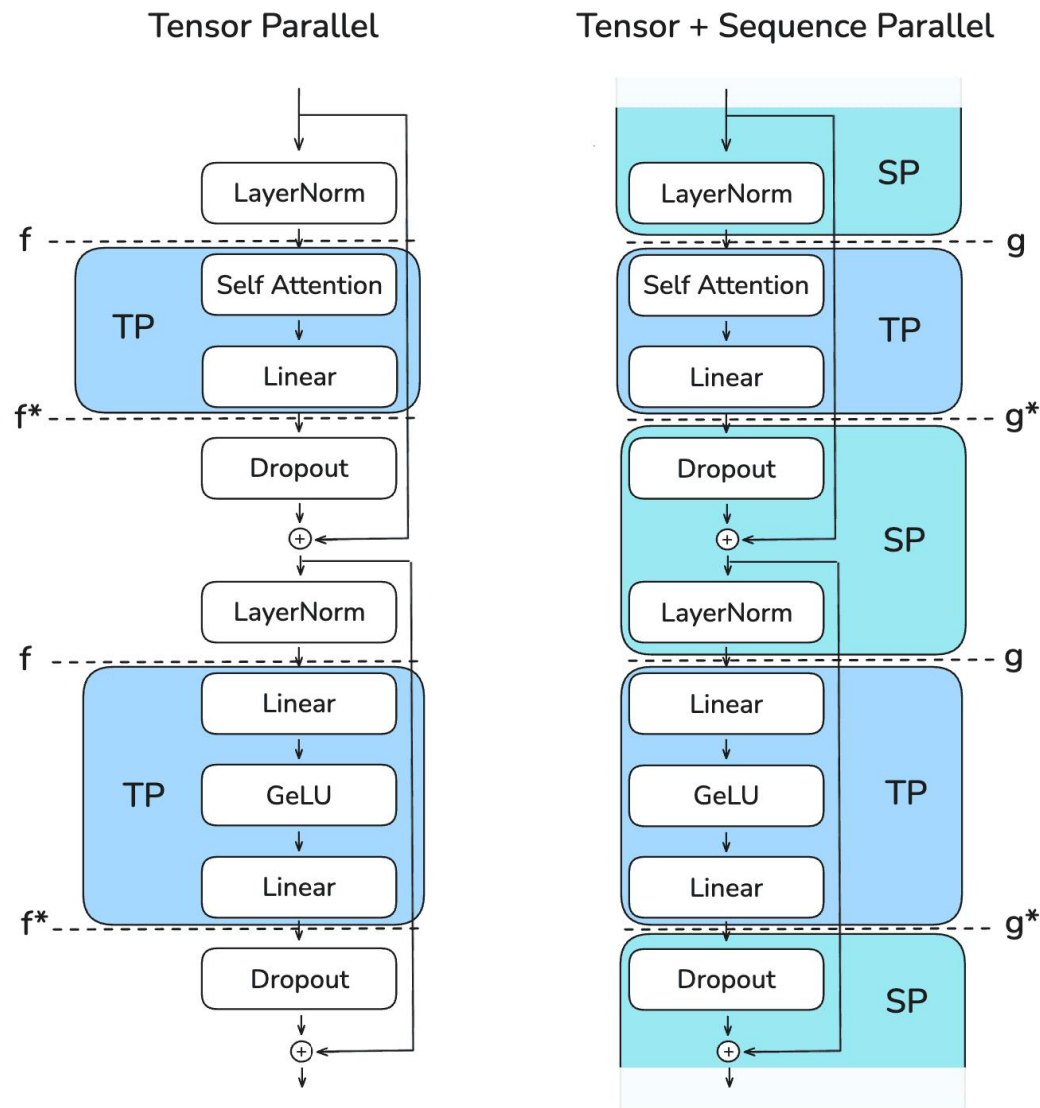
因此，尽管这些操作在计算上成本低廉，但它们仍然需要大量的激活内存。序列并行性允许通过沿序列维度分割来将内存负担分散到多个GPU上。

序列并行

序列并行性 (SP)

以下图表展示了如何使用不同的集体操作（标记为f和g）在张量并行和序列并行区域之间进行转换。在实践中，将从左到右进行：

关键挑战是在保持内存使用低的同时，高效地管理这些转换并保持正确性。



序列并行

在张量并行中

在正向传播中：

- ✓ f 是一个无操作（no operation），因为激活已经在各个rank之间复制。
- ✓ f^* 是一个All-Gather操作，用于同步激活并确保正确性。

而在反向传播中：

- ✓ f^* 是一个无操作，因为梯度已经在各个rank之间复制。
- ✓ f 是一个All-Gather操作，用于同步梯度。

这些 f 和 f^* 操作被称为共轭对，因为它们相互补充——在每个传播过程中，当一个操作是无操作时，另一个操作是All-Gather，而在另一个传播过程中则相反。

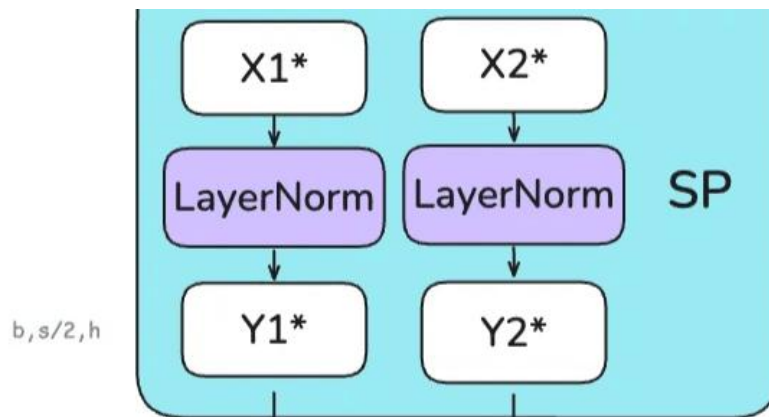
序列并行

对于序列并行性，使用标记为g和g*的不同操作

- ✓ 具体来说，避免在SP区域使用All-Gather，因为这需要收集完整的激活并增加的峰值内存使用，违背了SP的目的。
- ✓ 那么这里实际上发生了什么？正如一位著名的LLM所说，让一步一步来：

初始层归一化层（SP区域）

输入张量 $X1^*$ 和 $X2^*$ 进入 $(b, s/2, h)$ ，已在序列维度上分割。每个GPU独立对其序列块执行LayerNorm计算，得到 $Y1^*$ 和 $Y2^*$ 。



序列并行

第一次转换 ($SP \rightarrow TP$)

- g 操作 (*All-Reduce*) 将 $Y1$ 和 $Y2$ 合并回完整序列长度。
- 恢复 $Y(b, s, h)$ 由于列线性层需要完整的隐藏维度 h 。

第一线性层 (TP 区域)

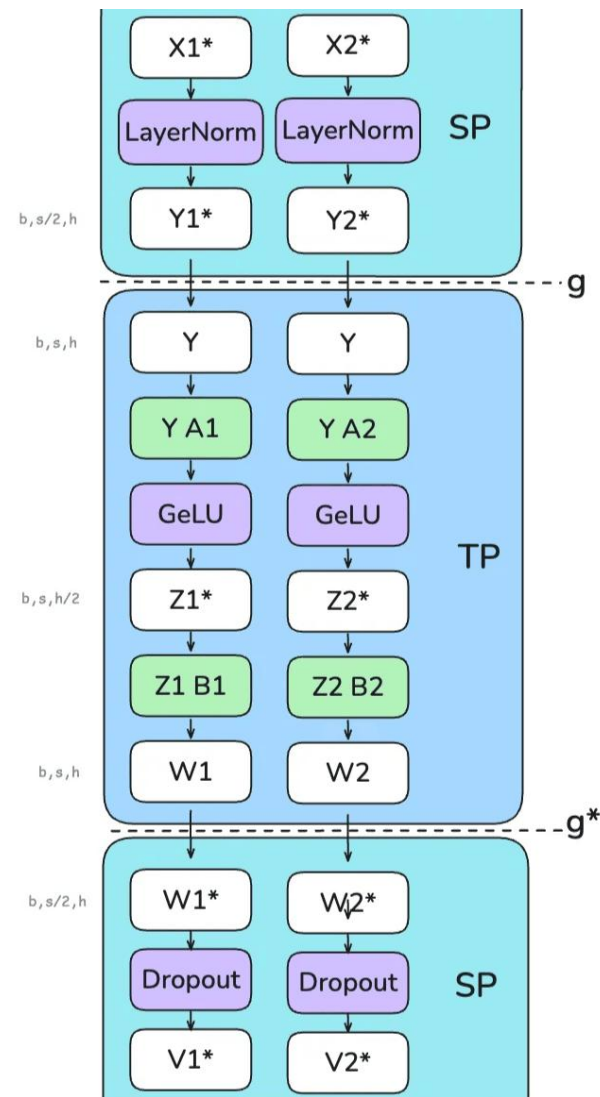
- $A1$ 和 $A2$ 是列线性层, 因此它们沿着隐藏维度分割 Y 。
- $GELU$ 在每个GPU上独立应用。 $Z1^*$ 和 $Z2^*$ 是 $(b, s, h/2)$ 。

第二线性层 (TP 区域)

- $B1$ 和 $B2$ 是行线性层, 因此它们恢复隐藏维度。
- $W1$ 和 $W2$ 是 (b, s, h) 需要相加。

最终过渡 ($TP \rightarrow SP$)

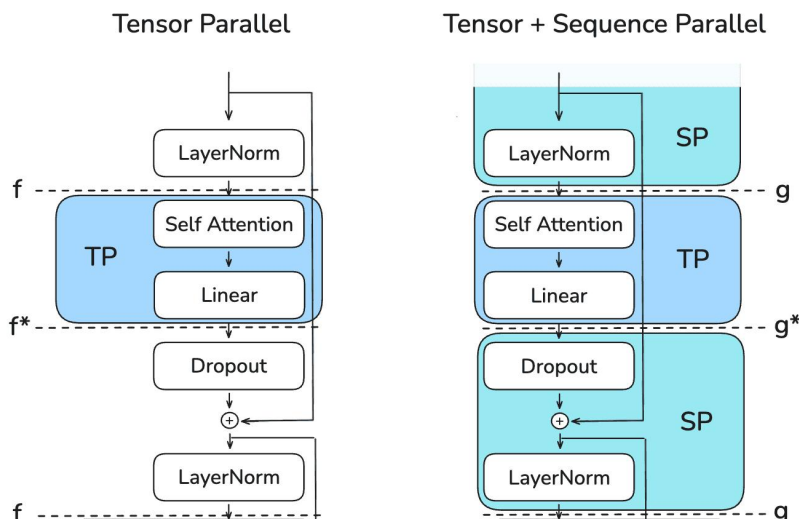
- g^* 操作 (减少-散射) 在序列维度上散射的同时, 减少了之前行线性层的正确性。 $W1^*$ 和 $W2^*$ 是 $(b, s, /2h)$ 。



序列并行

序列并行的一个关键优势

- ✓ 序列并行的一个关键优势是它减少了需要存储的最大激活大小。仅使用张量并行时，不得不在各个点存储形状为 (b, s, h) 的激活。
- ✓ 然而，使用序列并行后，最大激活大小减少到 $\frac{b \cdot s \cdot h}{tp}$ ，因为总是沿着序列或隐藏维度进行分割。



序列并行

总结在正向传播过程中，激活（又称隐藏状态）在隐藏维度 h 和序列维度 s 上的形状变化：

区域	仅TP	带有SP的TP
输入TP（列线性）	h : sharded (weight_out is sharded) s : full	h : sharded (weight_out is sharded) s : All-Gather to full
TP区域	h : sharded s : full	h : sharded s : full
退出TP（行线性）	h : full (weight_out is full + All-Gather for correctness) s : full	h : full (weight_out is full + Reduce-Scatter for correctness) s : Reduce-Scatter to sharded
SP区域	h : full s : full	h : full s : sharded

序列并行

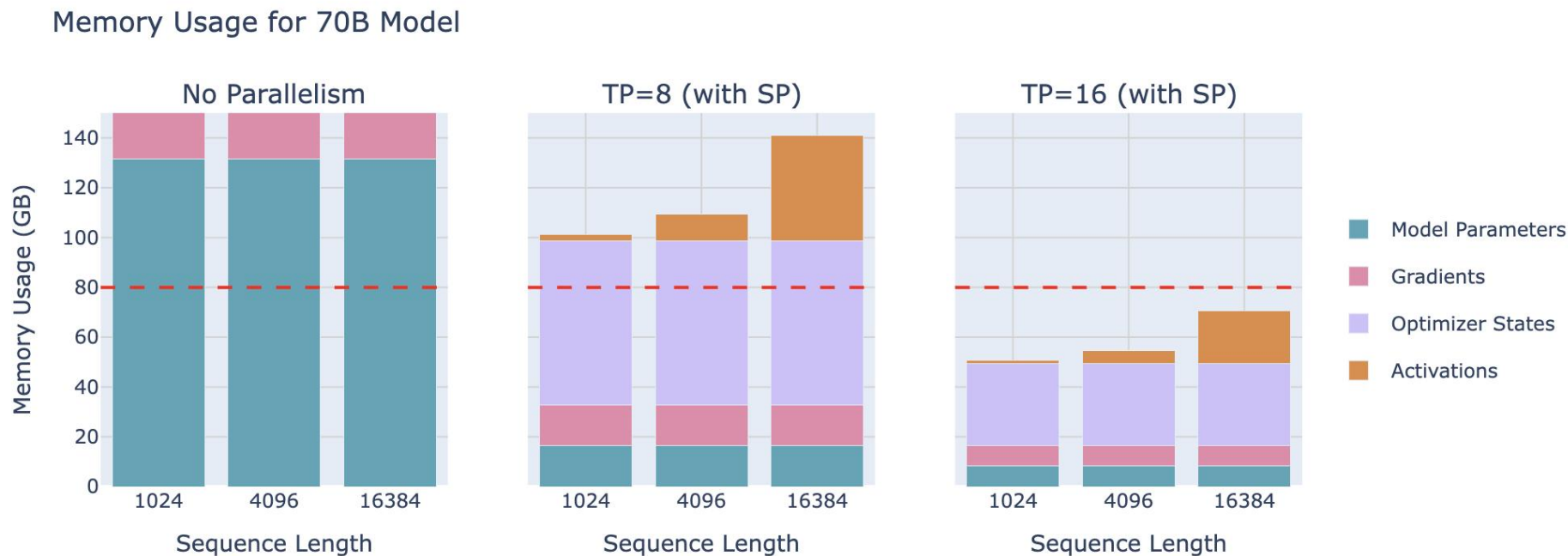
对于嵌入层：

区域	Vanilla TP	带有SP的TP
嵌入层（行线性，在词汇上分片）	h : full (weight_out is full + All-Gather for correctness) s : full	h : full (weight_out is full + Reduce-Scatter for correctness) s : Reduce-Scatter to sharded

序列并行

通过使用序列并行性

- ✓ 可以实现更大的激活内存节省，使能够将批处理大小和序列长度推得比单独使用张量并行性更远。让看看这对之前的70B模型示例意味着什么：

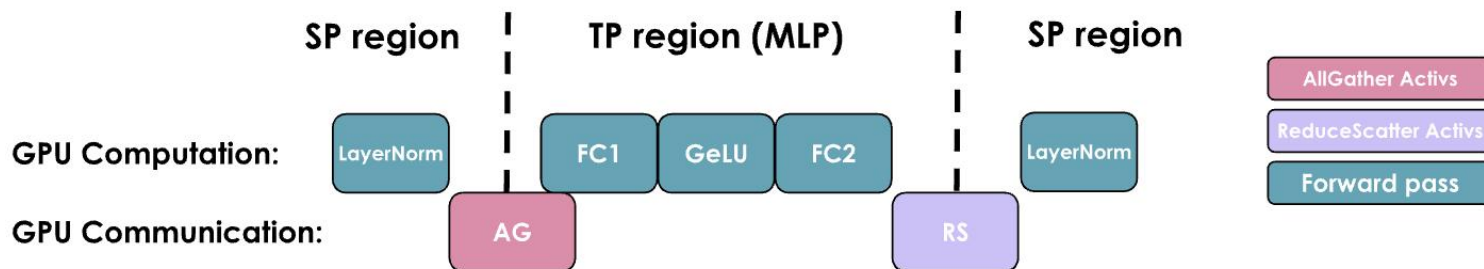


序列并行

使用TP+SP是否比原始TP产生更多的通信开销

- 是的，也不是。在原始TP的前向传递中，每个transformer块有两个All-Gather操作，而在SP中，每个transformer块有两个All-Gather和两个Reduce-Scatter操作。所以，SP的通信操作数量是TP的两倍。
- 但是，由于All-Gather操作可以被分解为All-Gather和Reduce-Scatter，它们在通信成本方面实际上是等效的。同样的推理也适用于反向传递，因为只是使用每个操作的共轭（no-op $\leftrightarrow$ All-Reduce和All-Gather $\leftrightarrow$ Reduce-Scatter）。

使用TP+SP时MLP分析



序列并行

就像TP一样，TP+SP难以与计算重叠

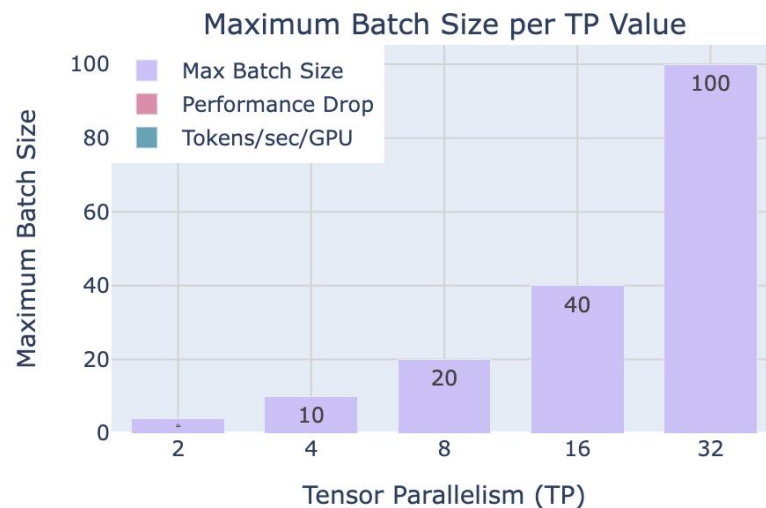
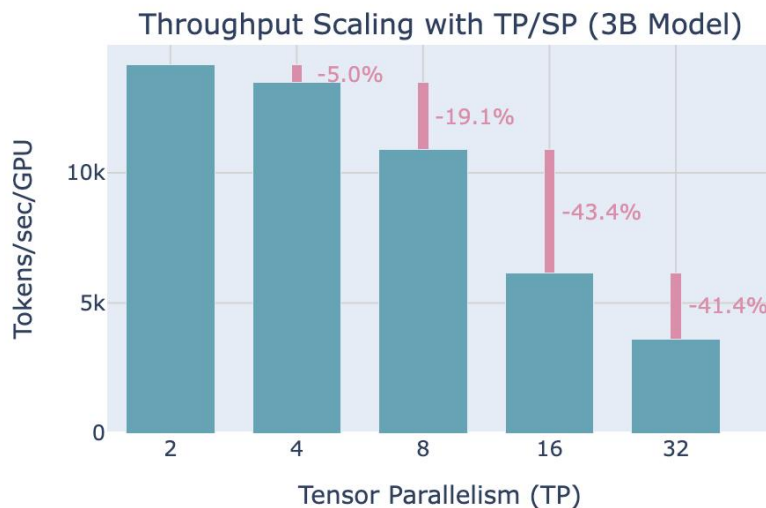
- 这使得吞吐量高度依赖于通信带宽。

就像TP一样，TP+SP通常只在节点内部进行

- （保持TP度低于每个节点的GPU数量；例如， $TP \leq 8$ ）。

可以通过扩大张量并行度来衡量这种通信开销如何变得越来越成问题。

让在将TP与SP一起扩大到3B参数模型（序列长度为4,096）时，测量吞吐量和内存利用率：



序列并行

就像TP一样，TP+SP难以与计算重叠

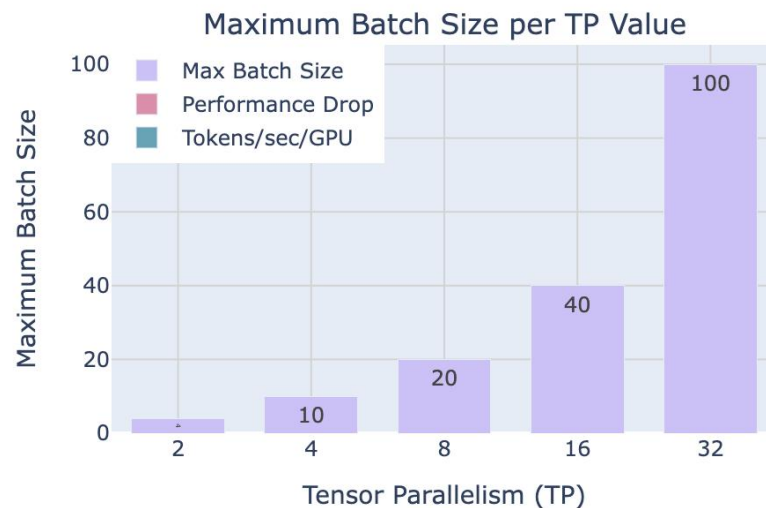
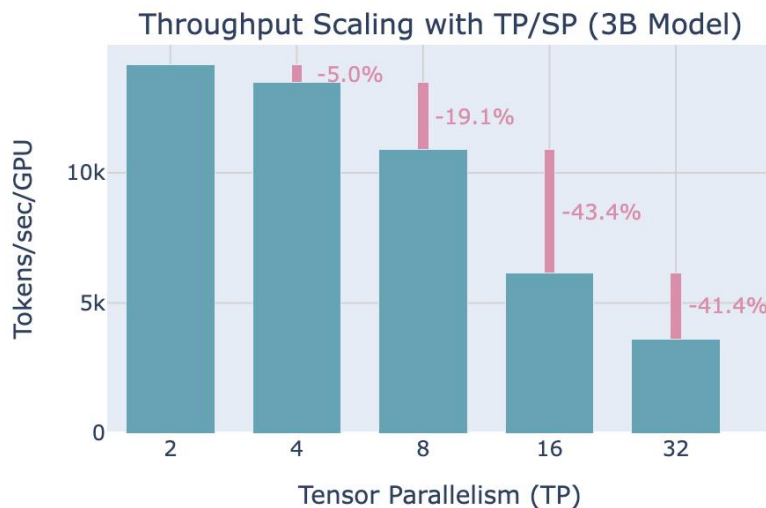
- 这使得吞吐量高度依赖于通信带宽。

就像TP一样，TP+SP通常只在节点内部进行

- （保持TP度低于每个节点的GPU数量；例如， $TP \leq 8$ ）。

可以通过扩大张量并行度来衡量这种通信开销如何变得越来越成问题。

让在将TP与SP一起扩大到3B参数模型（序列长度为4,096）时，测量吞吐量和内存利用率：



序列并行

计算效率（左侧）和内存容量（右侧）之间存在权衡

- 虽然更高的并行度可以通过减少激活内存来处理更大的批量大小，但它们也会降低每个GPU的吞吐量，尤其是在每个节点GPU数量对应的阈值以上。

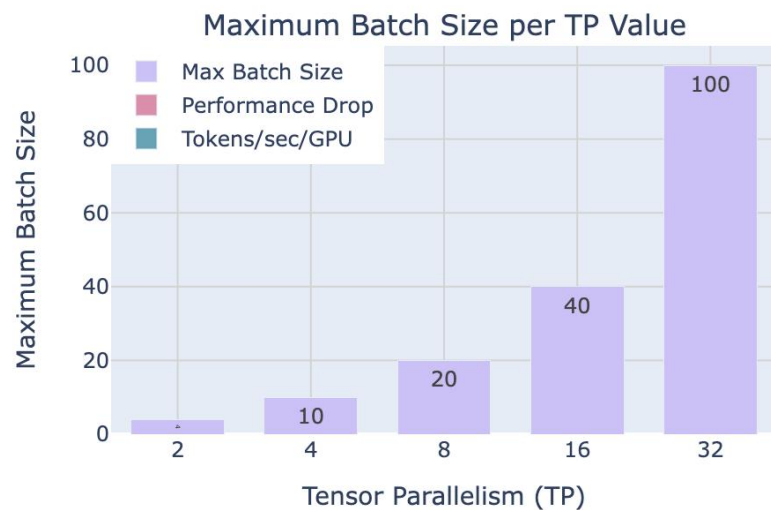
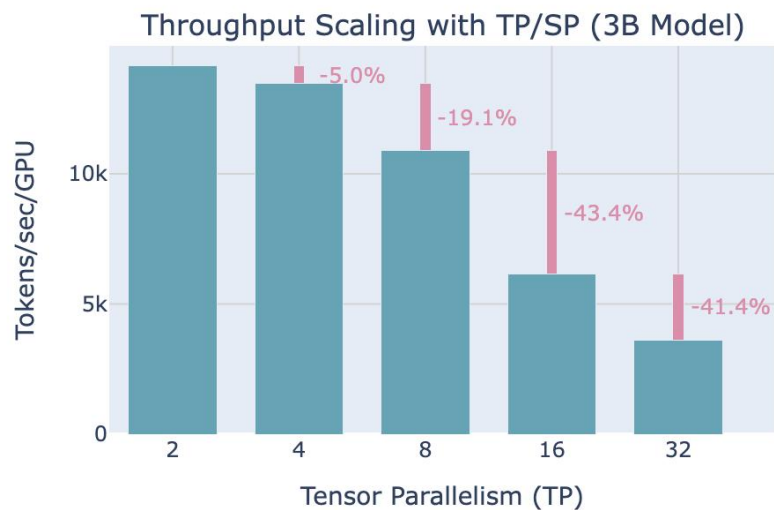
总结

- 对于这两种方法，注意到当从TP=8移动到TP=16时性能下降最大，因为那时是从仅在单个节点内通信（NVLink）转变为节点间的通信（EFA）。
- 使用TP与SP结合使用时，激活内存的节省帮助适应比仅使用TP时更大的批量。

序列并行

TP+SP有两个限制

- 如果扩展序列长度，激活内存仍然会在TP区域爆炸，如果模型太大而无法适应TP=8，将因为节点间的连接性而看到巨大的减速。
- 可以通过上下文并行处理第一个问题，通过流水线并行处理第二个问题。



序列并行

序列并行性 (SP)

序列并行性 (SP) 涉及将模型中由张量并行性未处理的部件的激活和计算进行拆分，例如dropout和LayerNorm，但沿着输入序列维度而不是隐藏维度进行拆分。

这是可行的

因为这些操作需要访问完整的隐藏维度以正确计算。例如，LayerNorm需要完整的隐藏维度来计算均值和方差：

$$\text{LayerNorm}(x) = \gamma \cdot \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta$$

其中 $\mu = \text{mean}(x)$ ， $\sigma^2 = \text{var}(x)$ 在隐藏维度 h 上计算。

因此，尽管这些操作在计算上成本低廉，但它们仍然需要大量的激活内存。序列并行性允许通过沿序列维度分割来将内存负担分散到多个GPU上。

目录

Contents

- 1 张量并行
- 2 上下文并行
- 3 流水线并行
- 4 专家并行
- 5 并行对比与扩展

上下文并行

使用张量并行和序列并行

可以显著降低每个GPU的内存需求，因为模型权重和激活都在GPU之间分布。然而，当在越来越长的序列上训练模型时（例如，当扩展到每个序列128k或更多标记时），仍然可能超过单个节点上的可用内存，因为仍然需要在TP区域内处理整个序列。

即使使用完整的激活重计算（这会带来大约30%的巨大计算开销）

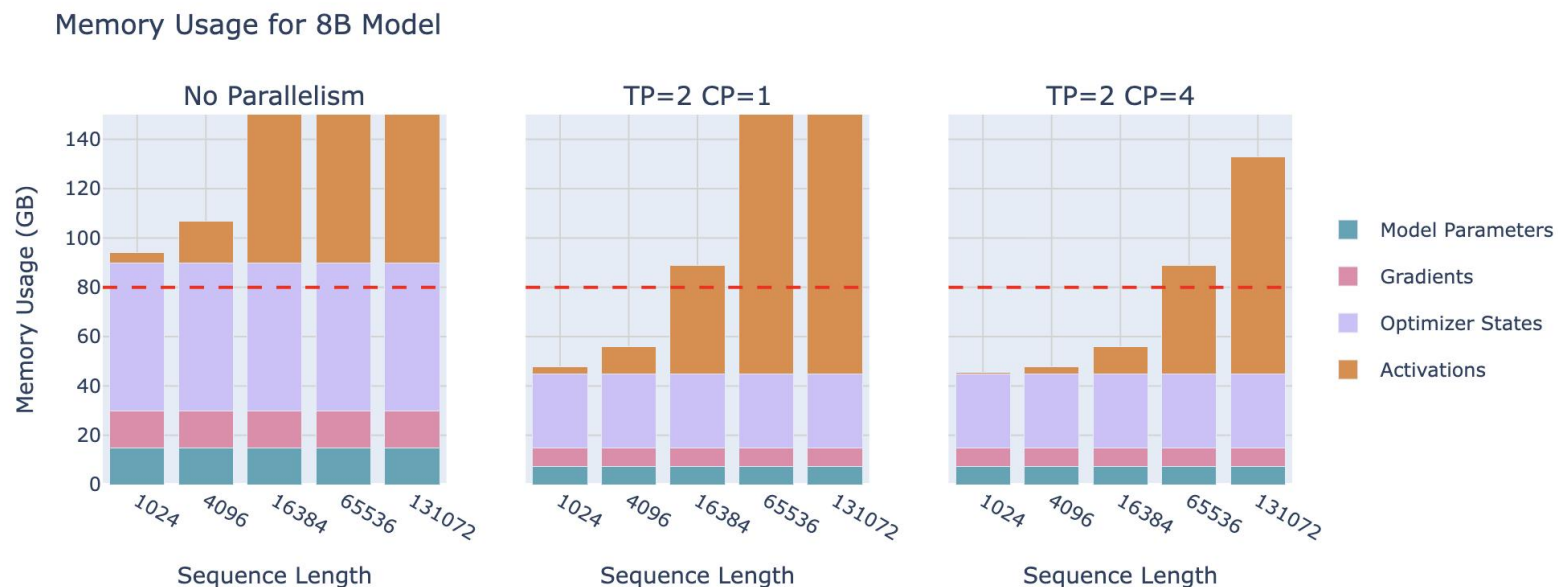
仍然需要在层边界处保留一些激活，这些激活的规模与序列长度成线性关系。进一步观察上下文并行（CP）如何起作用

上下文并行

上下文并行化

上下文并行化的核心思想与序列并行化（即沿序列长度分割）类似，但这种方法应用于已应用张量并行的模块。因此，将这些模块沿两个维度进行分割，从而也减少了序列长度的影响。

在上下文并行中，就像序列并行一样，在序列维度上拆分输入——但现在沿着整个模型应用这种拆分，而不仅仅是模型中序列并行区域，就像之前用TP+SP所做的那样。



上下文并行

拆分序列不会影响大多数模块

如MLP和LayerNorm，其中每个标记独立处理。它也不需要像TP那样昂贵的通信，因为只拆分了输入，而不是权重矩阵。就像数据并行一样，在计算梯度后，将启动一个All-Gather操作来同步CP组中的梯度。

尽管如此，有一个重要的例外

- 需要特别注意注意力块。在注意力模块中，每个标记需要访问所有其他序列标记的键/值对（或者，在因果注意力的情况下，至少关注每个前一个标记）。
- 由于上下文并行在GPU之间沿序列维度拆分输入，因此注意力模块将需要GPU之间进行完全通信以交换必要的键/值数据。
- 如果简单地这样做，这听起来非常昂贵。有没有更便宜、更快的方法呢？幸运的是，有一个核心技术能够有效处理键/值对的这种通信：环形注意力。

环形注意力

环形注意力

在这个注意力机制的实现中，每个GPU首先启动一个异步通信操作，将其键/值对发送到其他GPU。在等待其他GPU的数据时，它计算内存中已有数据的注意力分数。理想情况下，在完成计算之前，从另一个GPU接收下一个键/值对，允许GPU在完成第一次计算后立即开始下一轮计算。

让来举例说明。

- 假设拥有四个GPU和一个包含四个token的输入。最初，输入序列在序列维度上均匀分割，因此每个GPU将只拥有一个token以及其对应的Q/K/V值。
- 比如说，Q1、K1和V1分别代表第一个token的查询、键和值，它们位于第一个GPU上。注意力计算将需要四个时间步来完成。

环形注意力

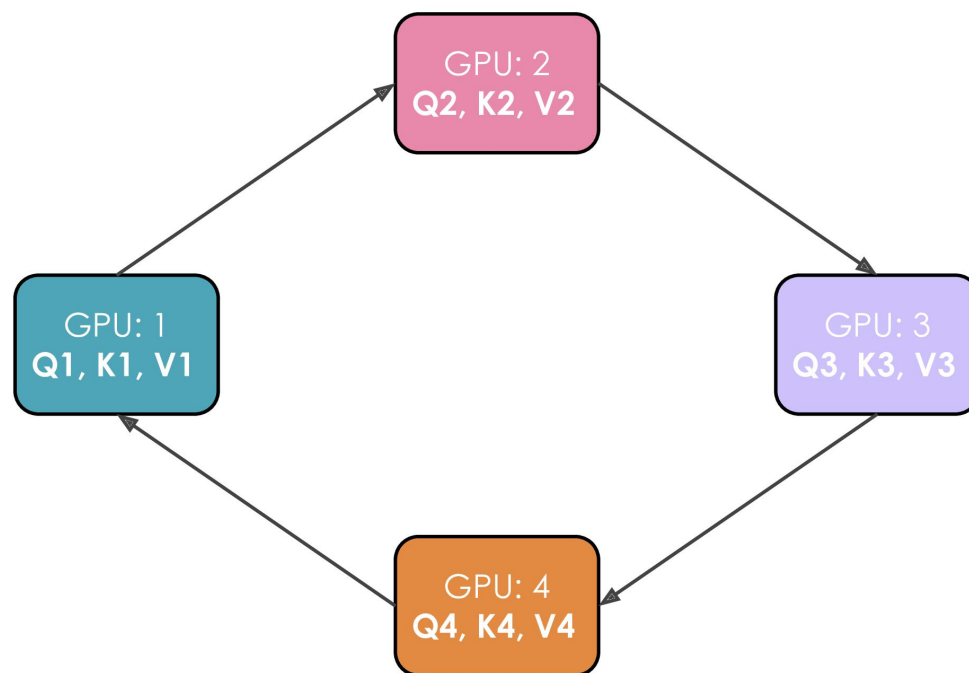
在每一个时间步，每个GPU执行这三个连续的操作：

1. 将当前键和值以非阻塞方式发送到下一台机器（除了最后一步之外），这样可以在前一个操作完成之前开始下一个操作。
2. 在本地计算当前键和值的注意力分数，这通常涉及执行 $\text{Softmax}\left(\frac{QK^T}{\sqrt{d}}\right) * V$ 。
3. 等待从上一个GPU接收键和值，然后回到步骤1，此时当前键和值是刚刚接收到的键/值。

执行这三个步骤四次以完成注意力计算。

环形注意力

以下动画展示了使用四个GPU的整个过程：



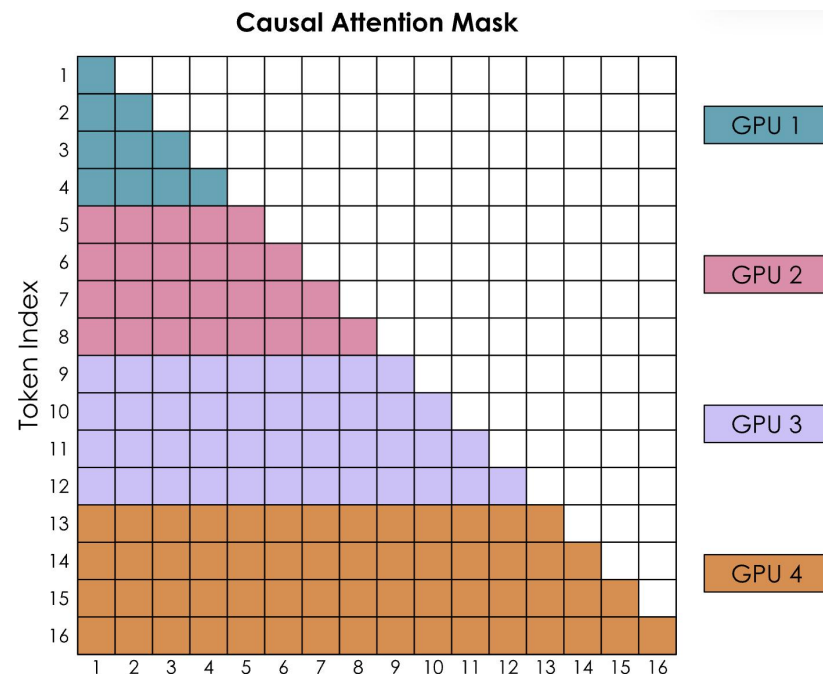
环形注意力

然而，有一个大问题

- 那就是由于因果注意力矩阵的形状，环状注意力的简单实现会导致GPU之间出现一些强烈的失衡。让通过考虑带有因果注意力掩码的注意力得分矩阵来查看softmax的计算：

softmax是按行计算的，这意味着每当GPU收到一行中的所有标记时，就可以进行计算

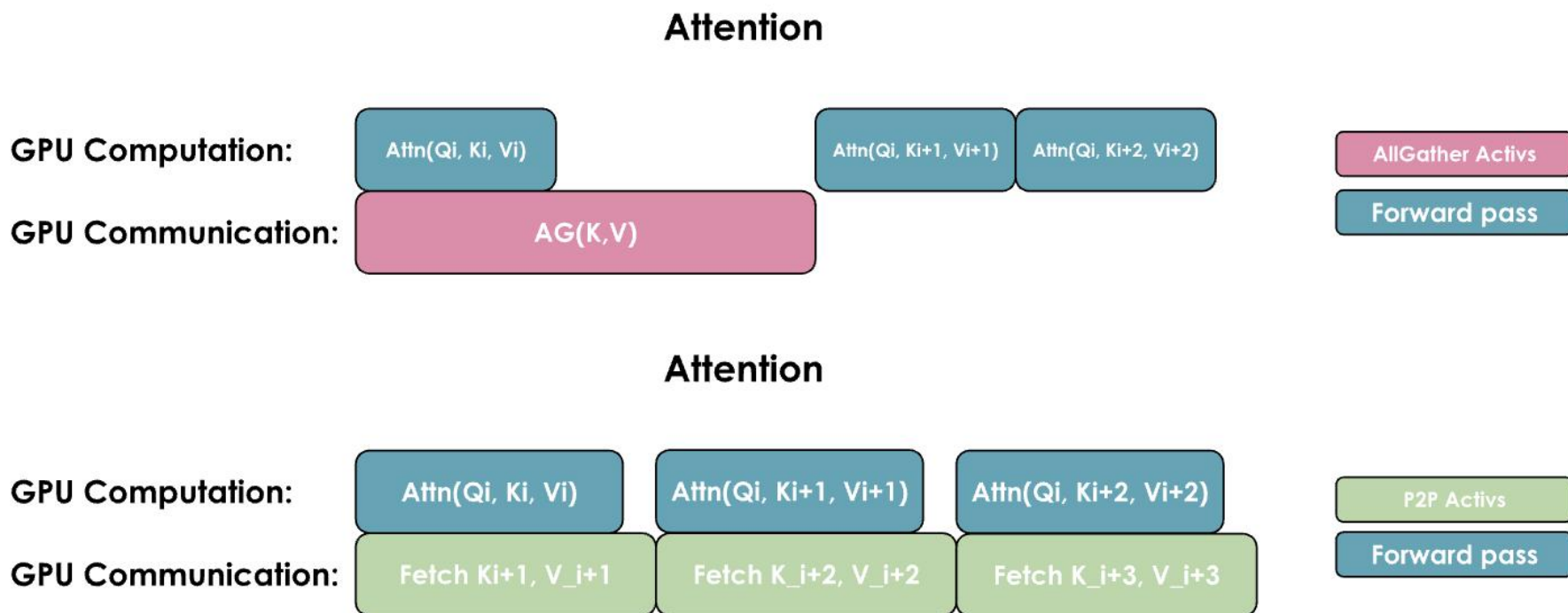
- 看到GPU 1可以立即计算它，因为它从标记1-4开始，不需要从任何其他GPU接收任何信息。然而，GPU 2将需要等待第二轮才能收到标记1-4，从而获得标记1-8的所有值。
- GPU 1似乎也比其他所有GPU完成的工作要少得多。



之字形环注意力 - 一种平衡的计算实现

有两种重叠计算和通信的通用方法

- 要么通过执行一个通用的全收集，同时在每个GPU上重新组合所有键和值（以ZeRO-3类型的方式），
- 要么根据需从每个GPU收集它们。



之字形环注意力 - 一种平衡的计算实现

这两个实现之间的关键区别在于它们的通信模式和内存使用：

1. All-Reudece实现：

所有GPU同时从所有其他GPU收集完整的键/值对。

需要更多的临时内存，因为每个GPU需要一次性存储所有K/V对。

通信一步完成，但内存开销更大。

2. 全对全（环形）实现：

GPU以环形模式逐块交换K/V对。

更节省内存，因为每个GPU只需要临时存储一个额外的块。

通信分散且与计算重叠，尽管由于多个通信步骤，存在一些额外的基本延迟开销。

全对全（环形）模式通常以略微复杂的通信模式为代价，提供更好的内存效率，而All-Reudece模式则更简单，但在注意力计算期间需要更多的临时内存。

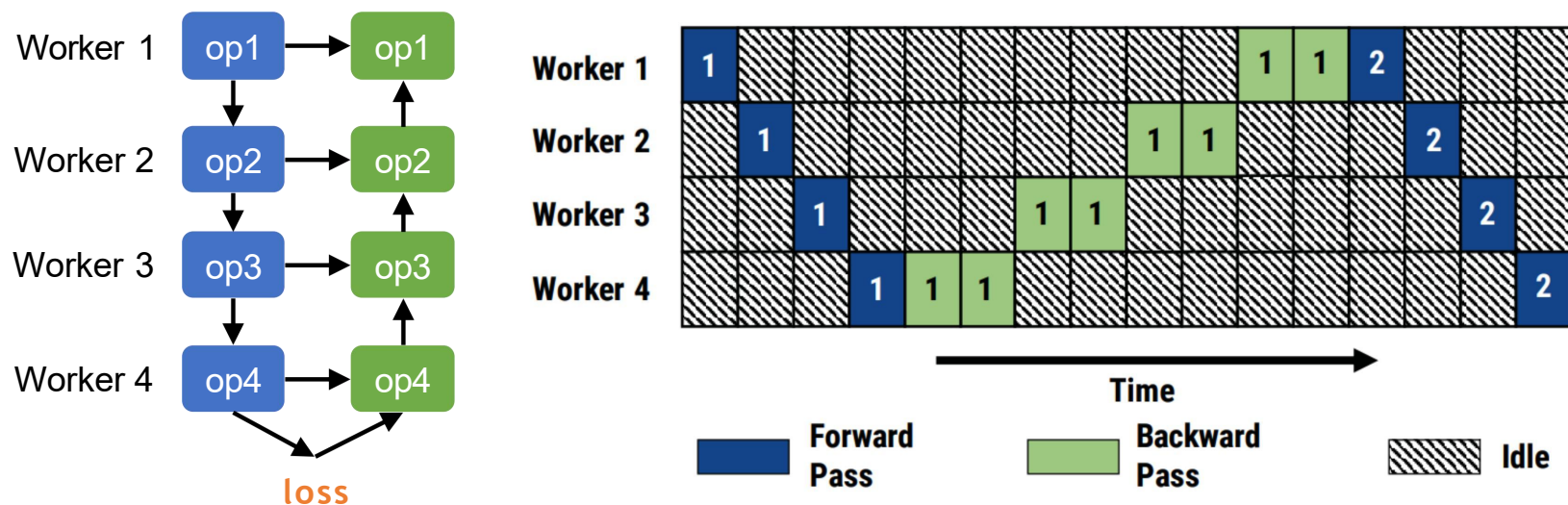
目录

Contents

- 1 张量并行
- 2 上下文并行
- 3 流水线并行**
- 4 专家并行
- 5 并行对比与扩展

模型并行性的问题

- 计算资源未充分利用
- 整体吞吐量低，由于资源利用率低



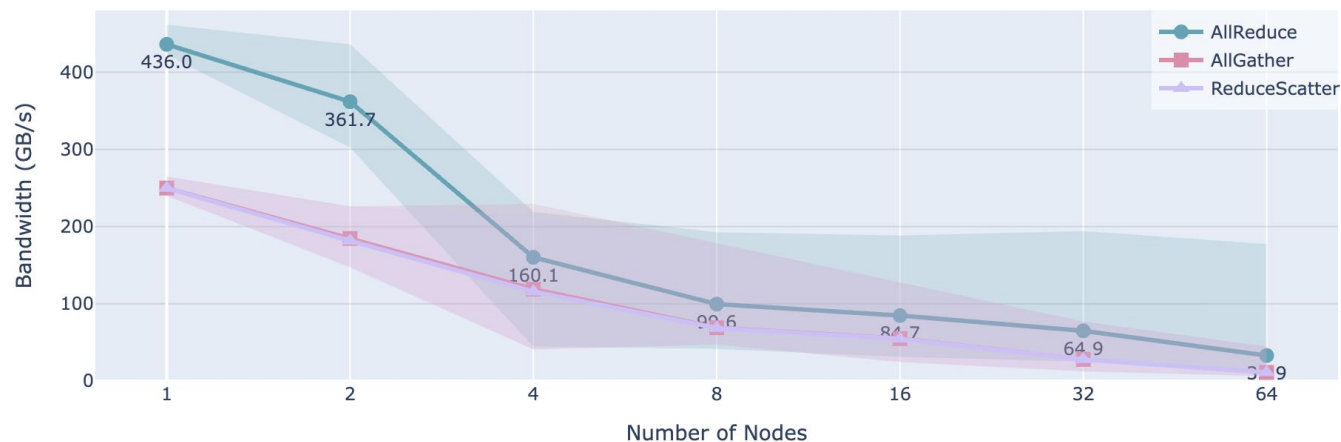
流水线并行

在“张量并行”部分

- 看到了尝试将张量并行扩展到单个节点上GPU数量（通常是4或8）之上，迫使使用低带宽的网络通信，这可能会显著影响性能。
- 当在跨多个节点的集群上（每个节点都有8个GPU）对其进行基准测试时，可以清楚地看到这种节点间通信的影响。

不同节点数量下的节点间通信带宽测量，显示所有减少、所有收集和减少分散操作的中位数（线条）以及5-95百分位范围（阴影区域）。

Communication Bandwidth by Number of Nodes (size=256MB)



流水线并行

序列和上下文并行对于长序列有帮助

- 但如果内存问题的根源不是序列长度，而是模型本身的大小，那么它们帮助不大。对于大型模型（70B+参数），仅权重的大小就可能已经超过了单个节点上4-8个GPU的限制。
- 可以通过召唤另一个并行维度来解决此问题：流水线并行（PP）。

流水线并行是一种简单但强大的技术——将模型层分散到多个GPU上

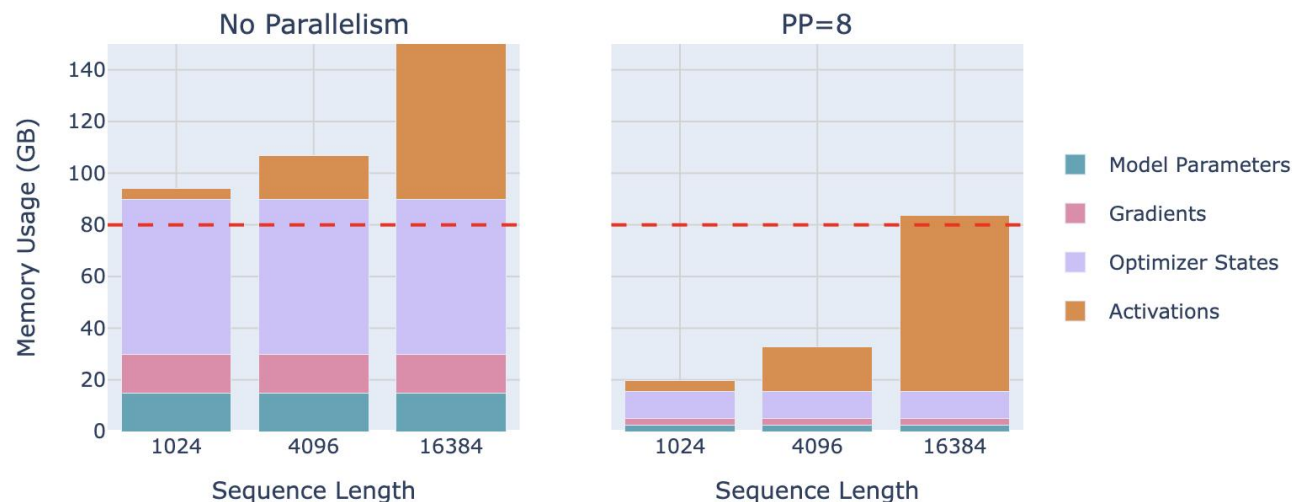
- 例如，如果有8个GPU，可以将层1-4放在GPU 1上，层5-8放在GPU 2上，以此类推。这样，每个GPU只需要存储和处理模型层的一部分，显著降低了每个GPU的内存需求。

流水线并行

让看看流水线并行在实际操作中对8B参数模型内存使用的影响：

一个有趣的现象：虽然模型参数在各个GPU上得到了很好的分配，但每个GPU上的激活内存保持不变！这意味着使用这种方法并没有节省任何激活内存。

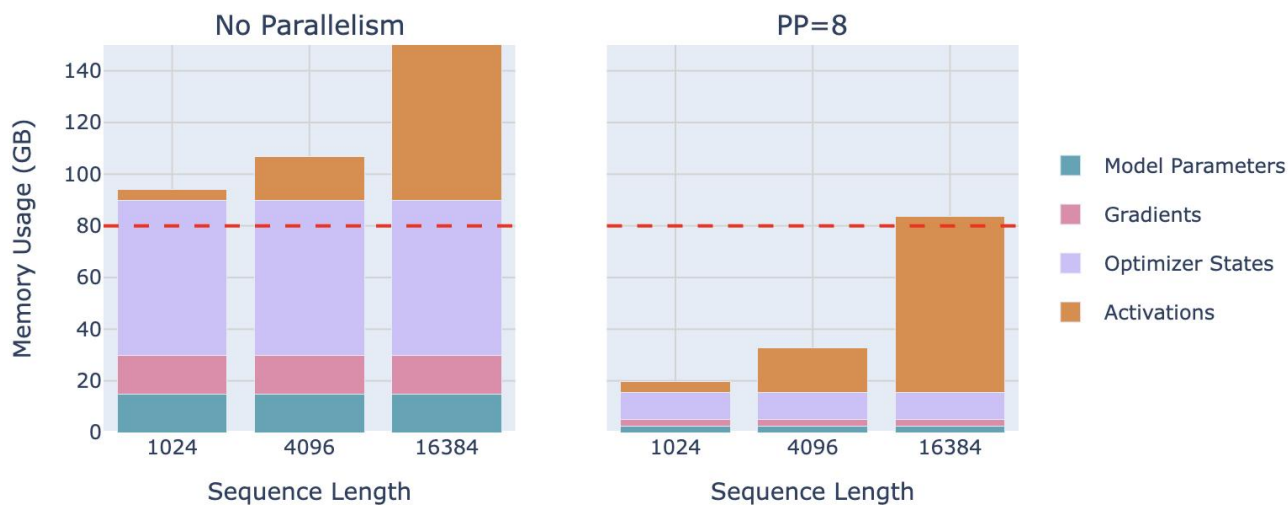
这是因为每个GPU在开始第一次反向传递之前需要执行PP前向传递。由于每个GPU处理1/PP层的部分，但在第一次反向之前需要处理PP微批次，因此最终存储 $PP \times (activs/PP) \approx activs$ ，这意味着激活内存需求与没有流水线并行处理时大致相同。



流水线并行

这引入了一种新的通信模式

- 与在数据并行中使用ZeRO-3进行通信的方式不同，现在在“流水线”中按顺序在GPU之间传递激活张量。虽然从概念上很简单，但有效地实现这项技术相当棘手。



流水线并行

首先，让假设只是将层分散到几个设备上

- 例如，第一个GPU将处理前几层，第二个GPU将处理模型的第二部分，依此类推。现在，通过的模型的前向传递现在简单地涉及按顺序将数据批次沿模型传递，从而依次使用每个计算设备。
- 第一大优势：所需的互连带宽保持相当低，因为只在模型深度的几个位置发送适度大小的激活。这与，例如，TP方法相比可以产生巨大的差异，在TP方法中，通信在每个层内发生多次。

流水线并行主义的主要挑战是如何有效地绕过PP的顺序性质

- 以保持的GPU始终忙碌，避免有一个GPU在计算而其他GPU在等待。

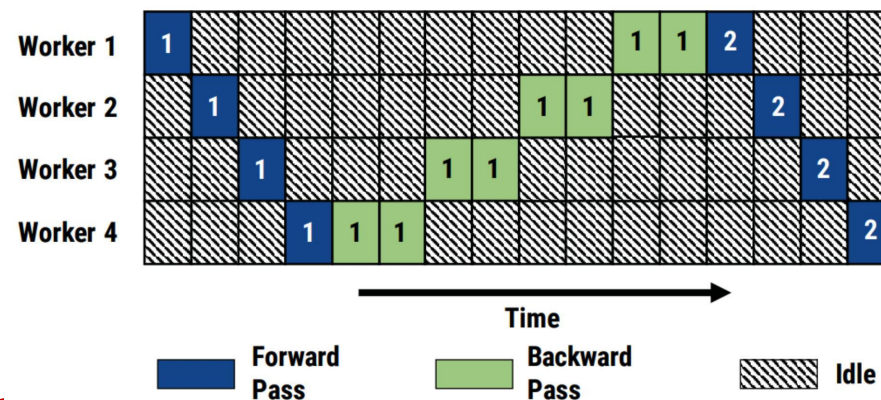
流水线并行

- **Mini-batch**: 每次迭代中处理的样本数

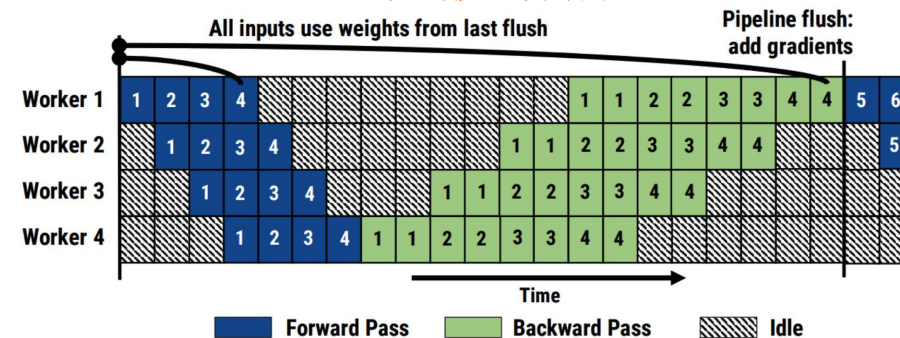
- 将一个小批次分成多个 **micro-batches**

- 跨微批次流水线前向和后向计算

模型并行

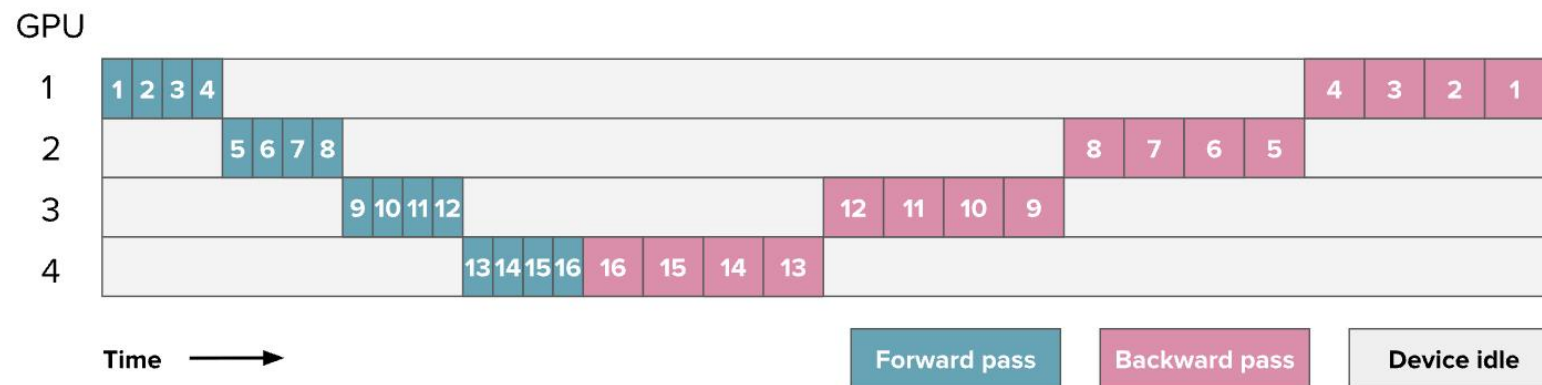


流水线模型并行性



流水线并行

以下是进行简单的前向和反向传递时的GPU利用率（这里，数字表示模型层）：



一个具有16层且分布在4个GPU上的模型的流水线并行示例。数字对应于层ID。

剩余的空闲时间以灰色表示，通常被称为“气泡”。

流水线并行

可以通过观察因为气泡而损失的时间来量化一个流水线设置的效率

- 假设 t_f 和 t_b 分别是正向和反向传递的时间，分别测量一个微批量和流水线的一个阶段（一个简单的假设通常是 $t_b \approx 2 \times t_f$ ，如上图所示）。
- 如果能够完美并行化，理想的总时间将是 $t_{id} = t_f + t_b$ 。然而，在这个例子中，由于流水线气泡，有额外的 $t_{pb} = (p - 1) * (t_f + t_b)$ 时间（其中 p 是流水线并行度；即 GPU 的数量）。这是每个 GPU 在其他 GPU 计算时等待的时间。

可以按照以下方式计算额外气泡时间与理想时间的比率：

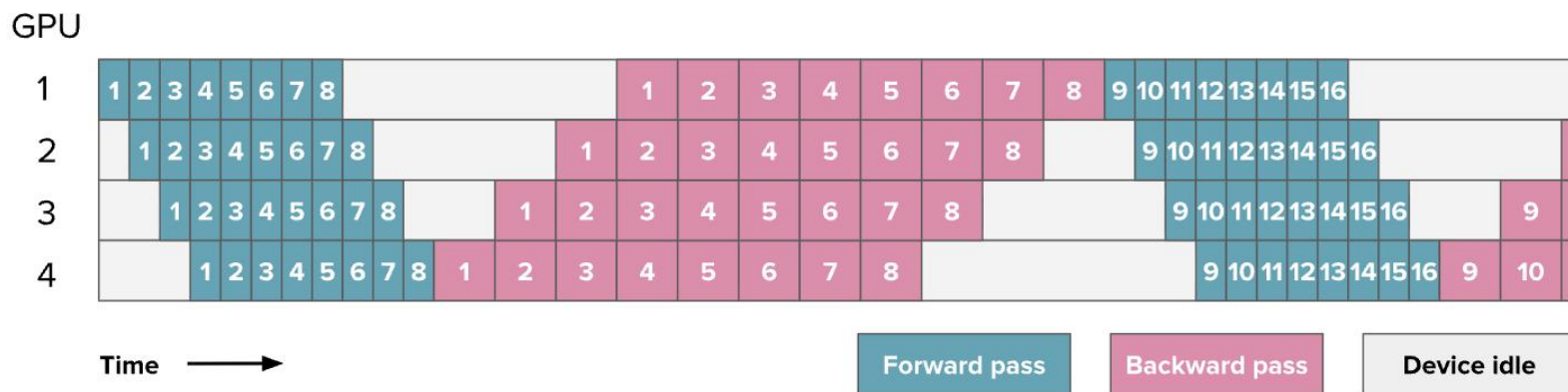
$$r_{bubble} = \frac{(p - 1) * (t_f + t_b)}{t_f + t_b} = p - 1$$

- 随着添加更多阶段,气泡时间因此增加,利用率下降。由此可见,在朴素的实施中,气泡可以很大!

流水线并行

考虑把批次拆分成更小的一口大小，可以并行（或几乎）处理

- 就像之前在 DP 方法中做的那样。现在，当第二块 GPU 忙于处理微批次 1 时，第一块 GPU 已经可以开始处理微批次 2 了。以下是使用八个微批量的计划：

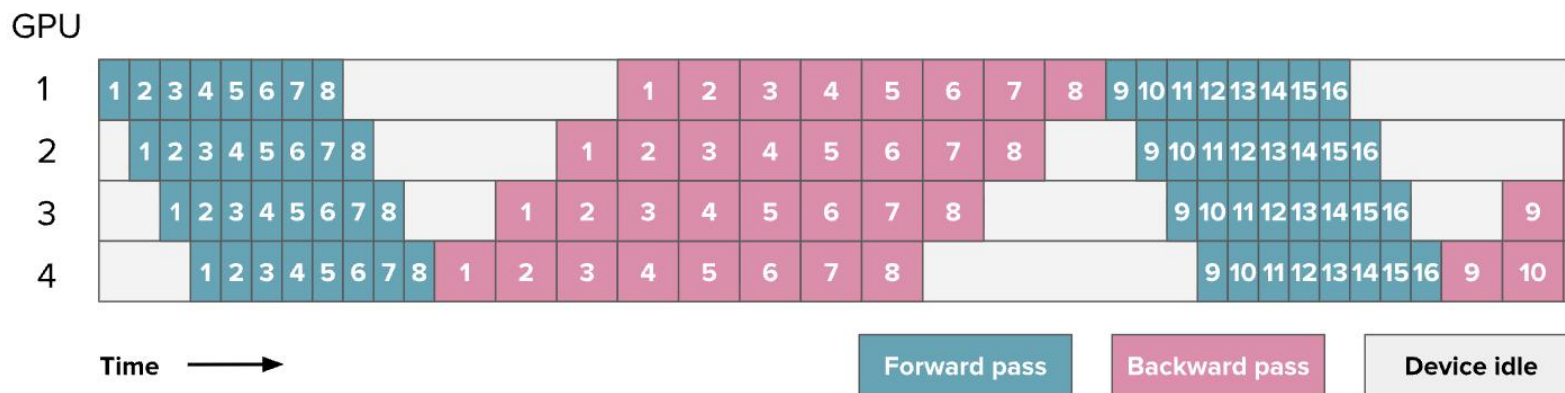


之前，图中的数字表示层，但从此所有流水线并行图中，数字表示微批次。可以把这里的每个方块看作包含多个层

流水线并行

全前后 (AFAB) 计划

- 先完成所有前传，然后再做所有后传。优点是前进和后退步骤通常是顺序的，因此保留了模型训练代码的整体组织。这种 PP 实现是最简单的实现之一。



让来看看 m 这个例子中的泡泡。

与第一个例子的区别在于，处理 m 个微批次的理想时间现在 $t_{id} = m * (t_f + t_b)$ 是：

$$r_{bubble} = \frac{(p-1) * (t_f + t_b)}{m * (t_f + t_b)} = \frac{p-1}{m}$$

流水线并行

全前后 (AFAB) 计划

如所见，可以通过增加更多微批次来减少 m 流水线阶段的低效，从而将气泡大小缩小 m 倍。

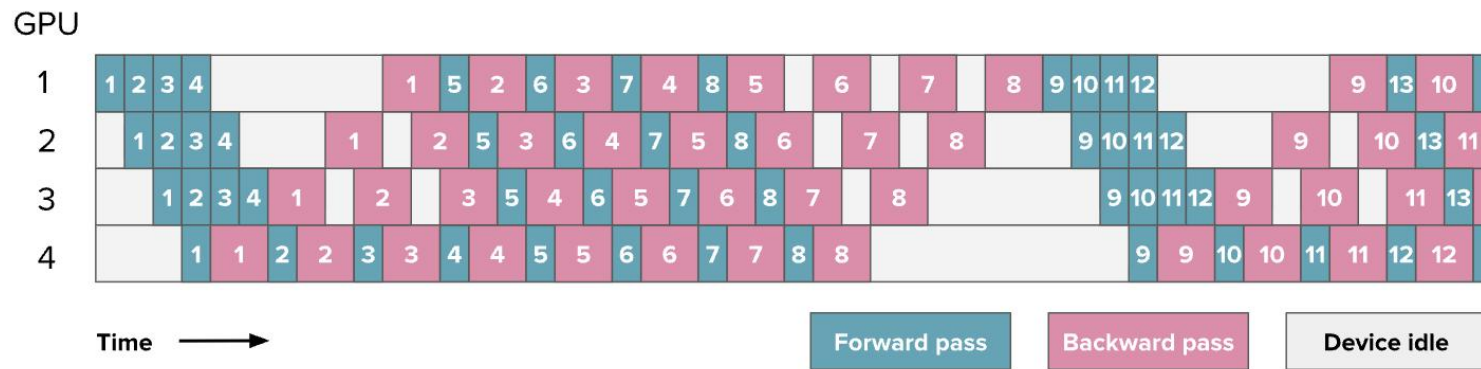
不过，和气泡一样烦人的还有存储所有激活所需的内存。

- 需要将所有激活都保存在内存中，直到进入反向传递阶段，这会迅速导致这些 PP 实现中的内存爆炸。
- 由于记忆爆炸是由存储的反向传递激活触发的，试试在还在进行前向计算时开始执行反向传递。这样就能尽快放弃一些反向传递所需的激活。

流水线并行

“一前一后” (1F1B)

中间/稳态涉及交替执行一次前传和一次后传。总体思路是尽快开始进行后传。时间表如下：



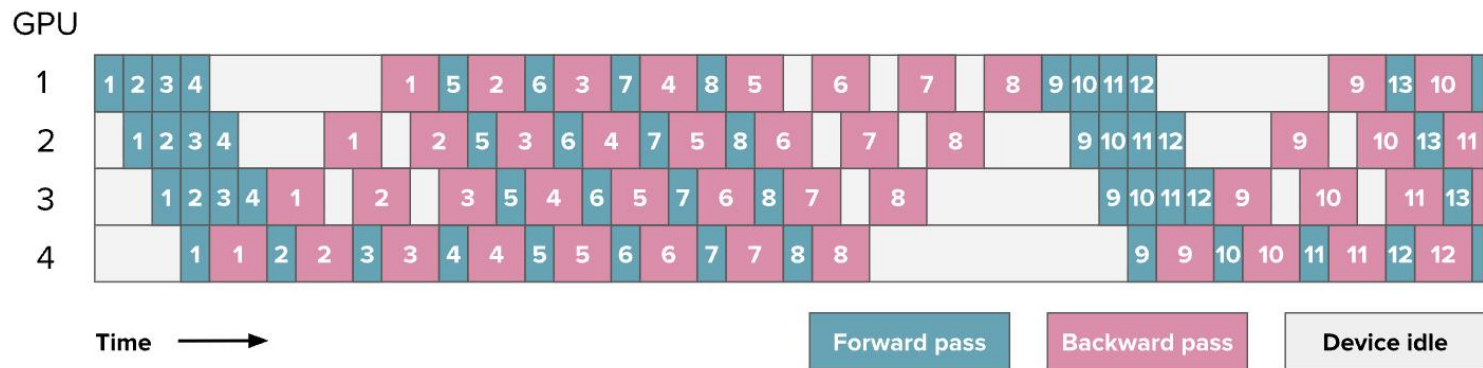
如果仔细数，会发现气泡仍然保持着相同大小，所以的训练效率并没有显著提升。然而，只需要存储 p 个微批量（ p 是流水线并行度），而非 m （ m 是微批次数），这可以减少 *AFAB* 计划中激活内存爆炸的情况。因此，可以添加更多的微批量，这样气泡实际上会减少。

流水线并行

“一前一后” (1F1B)

该设置的一个主要复杂性，如图所示，是前向和后向传递不再是干净顺序，而是在设备间并行执行并交错进行。这意味着必须在每个设备上独立安排前向到后向的切换，而不是像往常那样简单且常见的中央训练循环。

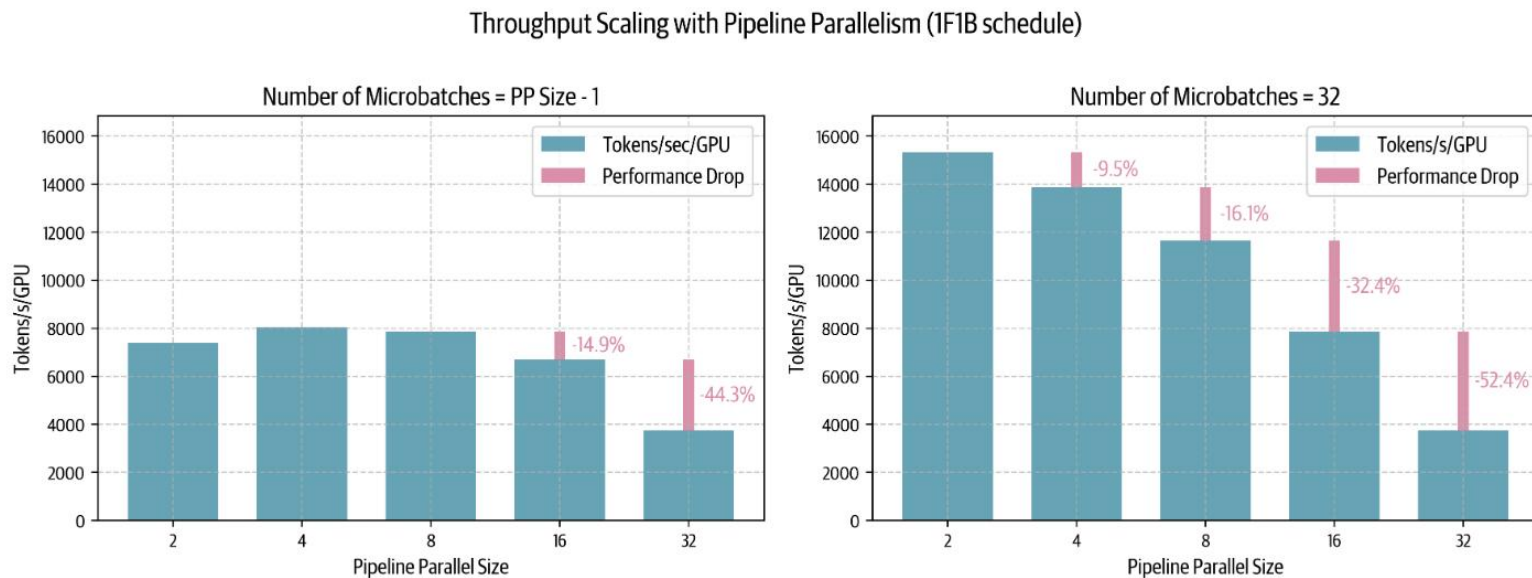
这也是实现流水线并行性通常需要对训练代码和建模代码进行较大修改的原因之一。



流水线并行

让看看 1F1B 流水线并行进度在实际中的扩展，并结合集群上的一些基准测试

左侧，当微批次数量等于或小于 PP 度减去 1 时 $m = p - 1$ ，可以看到流水线泡沫的有害性——性能低下，甚至随着 PP 的缩放而下降。

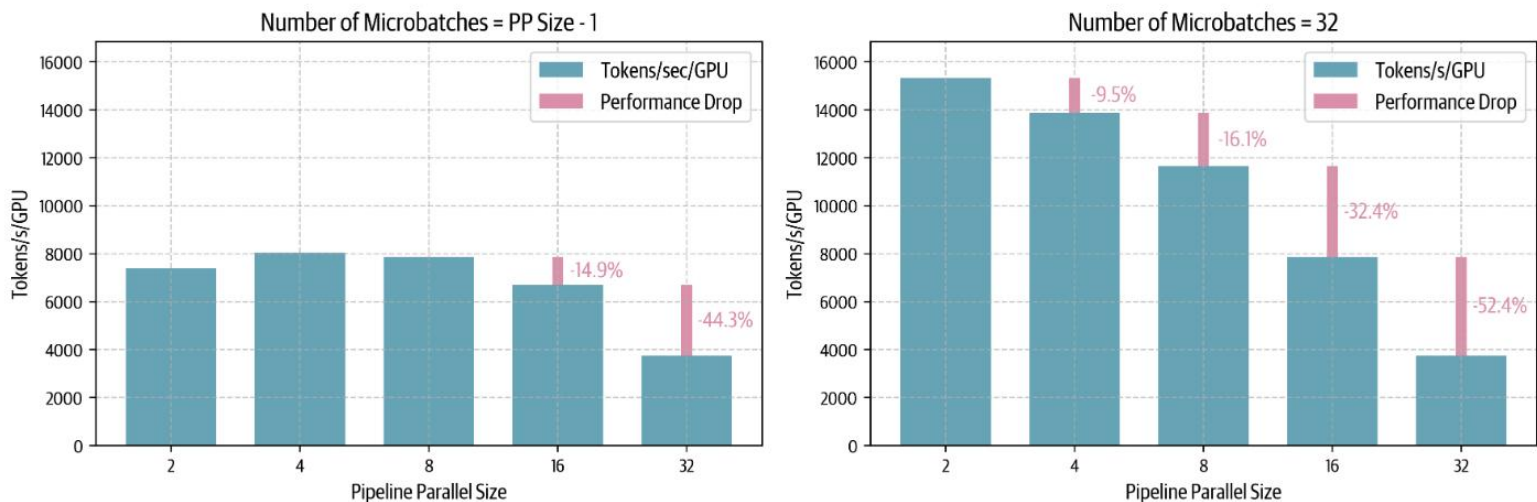


流水线并行

让看看 1F1B 流水线并行进度在实际中的扩展，并结合集群上的一些基准测试

右图显示，使用远多于 PP 度($m = 32 \gg p - 1$) 的微批量有助于提升低 PP 度的性能，尽管在非常大的 PP 度下仍有限制。实际上，无法任意增加微批量数量以维持比例 $m \gg p - 1$ ，因为最终受限于目标全局批次大小。随着 PP 度数增加，微批次数量达到最大，最终必须根据 $r_{bubble} = \frac{p-1}{m}$ 来增加气泡大小。

Throughput Scaling with Pipeline Parallelism (1F1B schedule)

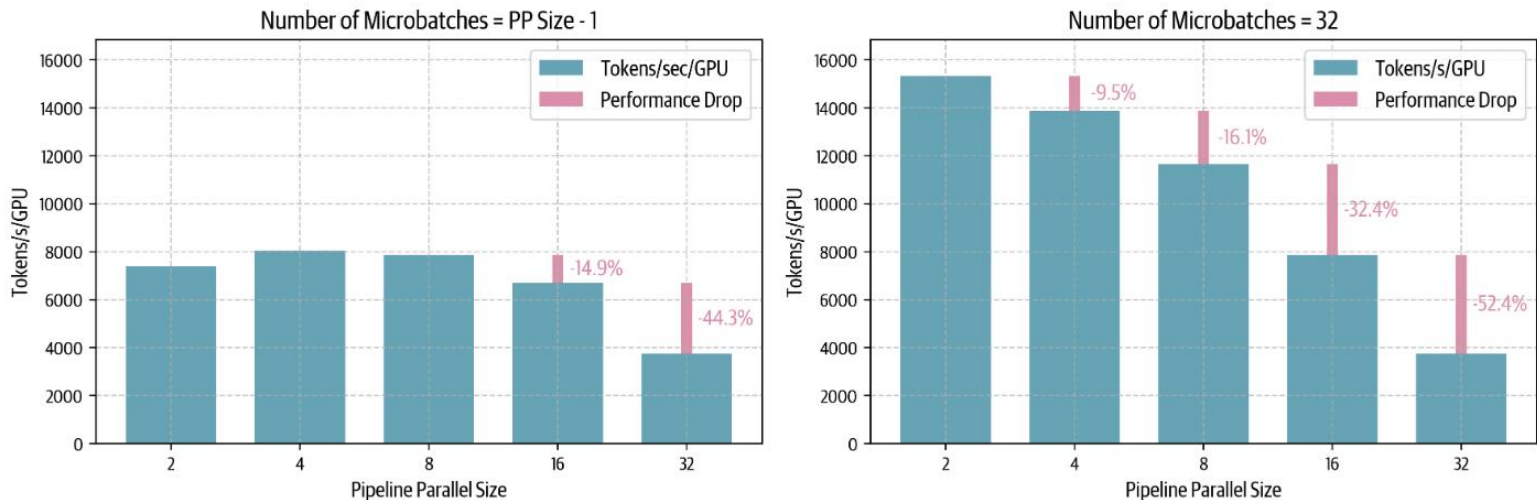


流水线并行

让看看 1F1B 流水线并行进度在实际中的扩展，并结合集群上的一些基准测试

有趣的是，在少量微批次中，从一个节点（ $p = 8$ ）扩展到两个节点（ $p = 16$ ）时性能仅下降 14%——这比用张量并行处理时表现要好得多，后者在类似跨节点场景下通常会降低约 43% 的性能。这种行为在低带宽节点间网络中表现得尤为吸引流水线并行性，适用于跨多节点的分布式训练。

Throughput Scaling with Pipeline Parallelism (1F1B schedule)

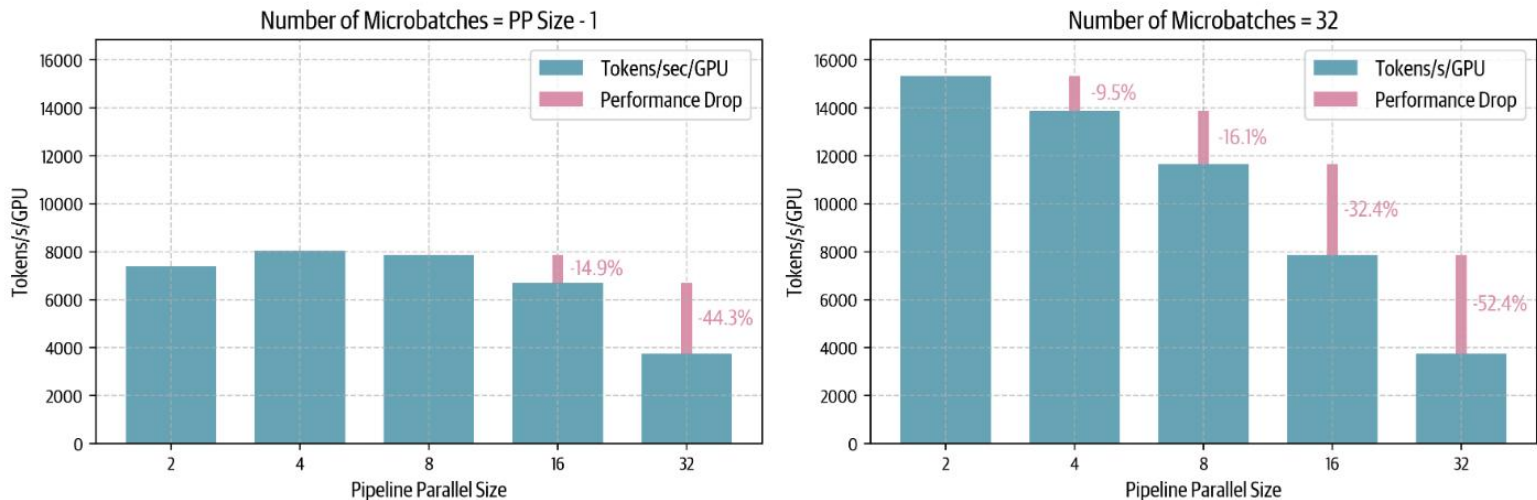


流水线并行

让看看 1F1B 流水线并行进度在实际中的扩展，并结合集群上的一些基准测试

虽然 1F1B 显著减少了激活记忆的占用，但从最后一张图中可以看到，流水线气泡依然是主要的效率瓶颈。由于气泡大小仍与流水线阶段数成正比，就让宝贵的 GPU 计算闲置。能否设计出更智能的计划，最大限度地减少这些浪费的计算时间？

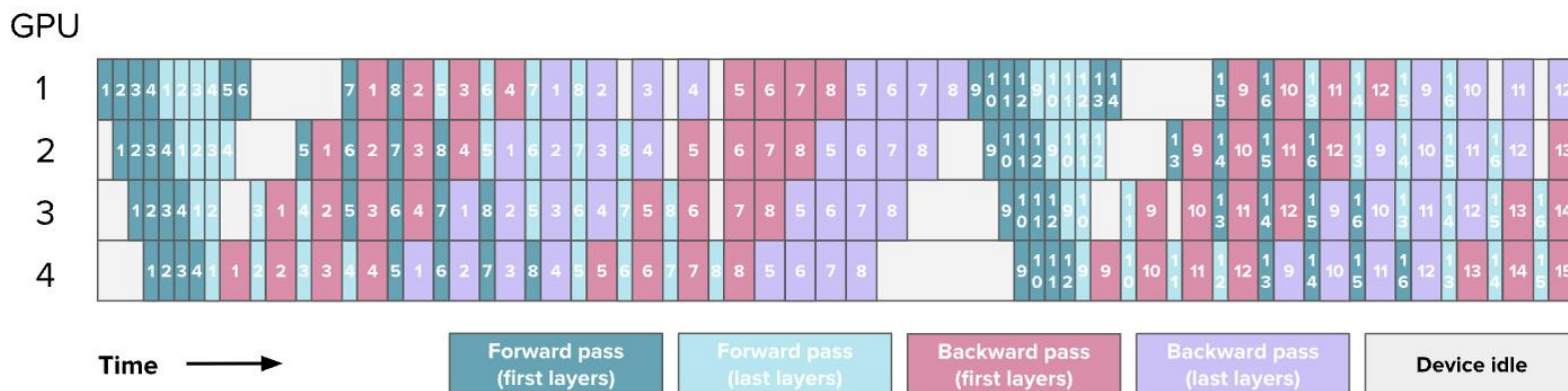
Throughput Scaling with Pipeline Parallelism (1F1B schedule)



流水线并行

交错阶段

- 到目前为止，一直简单地沿着模型深度维度切片模型，例如第一块 GPU 上放置第 1-4 层，在第二块 GPU 上托管第 5-8 层。但还可以考虑其他切片层的方法，比如第一块 GPU 用奇数层（1、3、5、7），在第二块 GPU 用偶数层（2、4、6、8）。
- 这通常可以看作是一种“循环流水线”，微批次在模型前向传递过程中会在一个 GPU 之间绕圈移动到另一个。让来看看这是如何运作的：



流水线并行

交错阶段

这里需要额外的通信，因为模型会对每个 GPU 的同一 v 计算进行时间，而之前 v 只进行了一次处理。然而，每次前向和后向传递都被 v 的因子除以，其中 v 是 GPU 的阶段数或模型块数，因为可以更好地交错前向和后向传递：

$$t_{pb} = \frac{(p-1) * (t_f + t_b)}{v}$$
$$r_{bubble} = \frac{1}{v} \frac{(p-1) * (t_f + t_b)}{m * (t_f + t_b)} = \frac{p-1}{v * m}$$

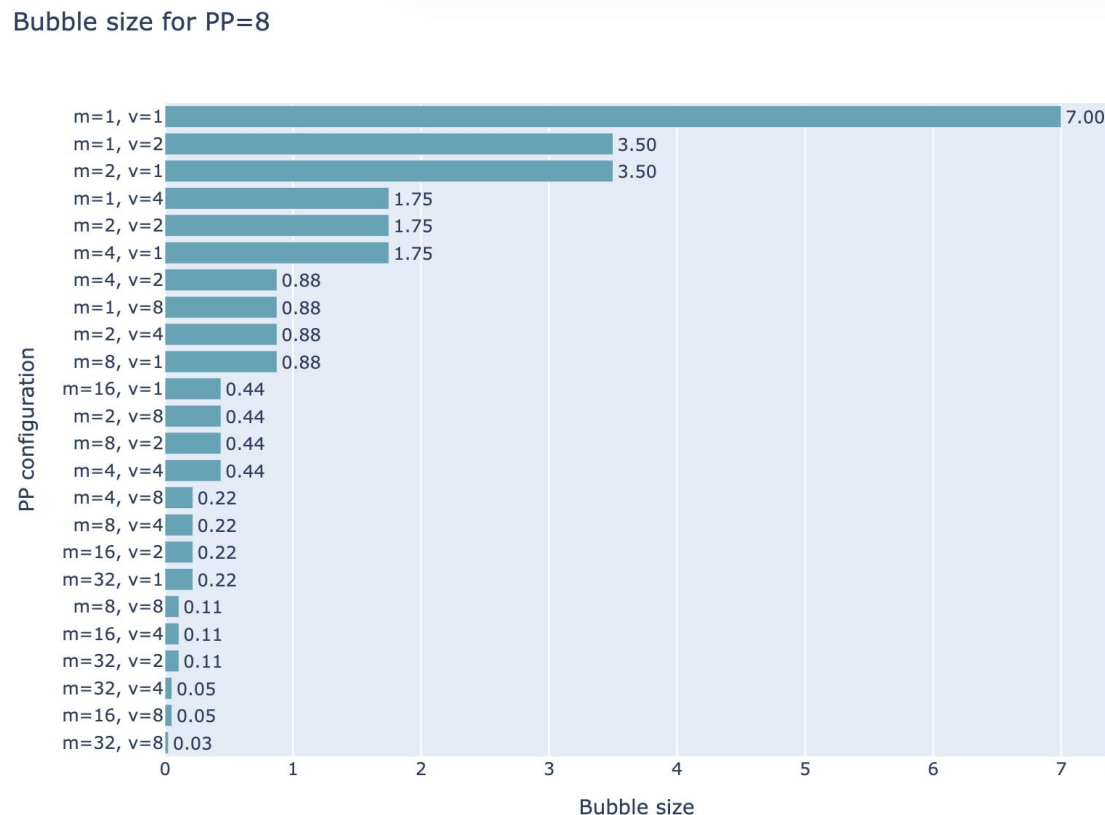
现在可以通过添加微批量和间差来减少气泡

- 但要注意，从量化上看，通信量也以 v 增加，所以这是权衡。在下图中，可以看到 PP 设置 $p = 8$ 的几种配置，其中特例对 $m = 1, v = 1$ 应于朴素流水线并行性，配置 $v = 1$ 为 AFAB 或 1F1B 配置， $v \neq 1$ 情况为交错配置。

流水线并行

现在可以通过添加微批量和间差来减少气泡

- 但要注意，从量化上看，通信量也以 v 增加，所以这是权衡。在下图中，可以看到 PP 设置 $p = 8$ 的几种配置，其中特例对 $m = 1, v = 1$ 应于朴素流水线并行性，配置 $v = 1$ 为 AFAB 或 1F1B 配置， $v \neq 1$ 情况为交错配置。



流水线并行

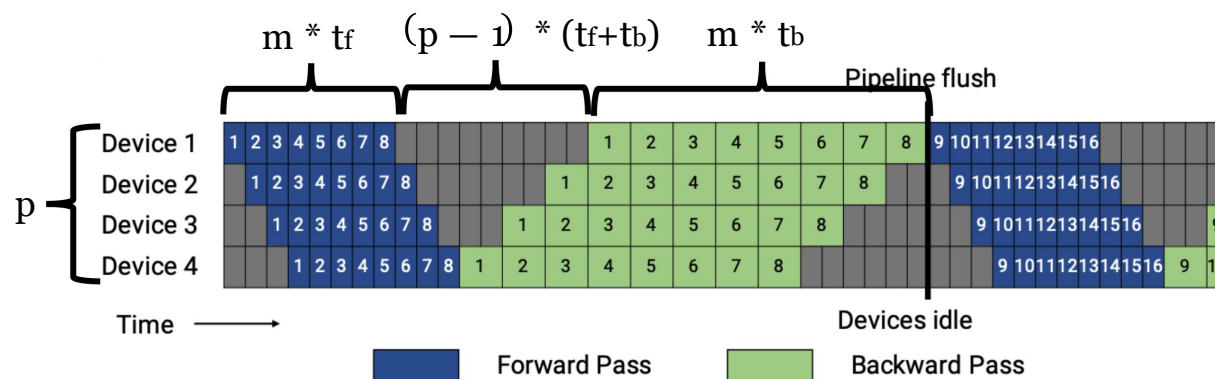
调度在这里也变得更加复杂

- 因为必须决定在给定的GPU和特定时刻，是优先处理通过后续层的早期微批次——这意味着尽可能快地关闭正向和反向循环（“深度优先”方法，该方法优先考虑尽可能快地将批次从模型中取出）——还是优先处理通过早期层的后续微批次（“广度优先”方法，该方法优先考虑尽可能多地填充流水线）。这一选择在“广度优先流水线并行性”论文中有详细解释。

“广度优先流水线并行性”论：<http://arxiv.org/pdf/2211.05953.pdf>

流水线模型并行性:设备利用率

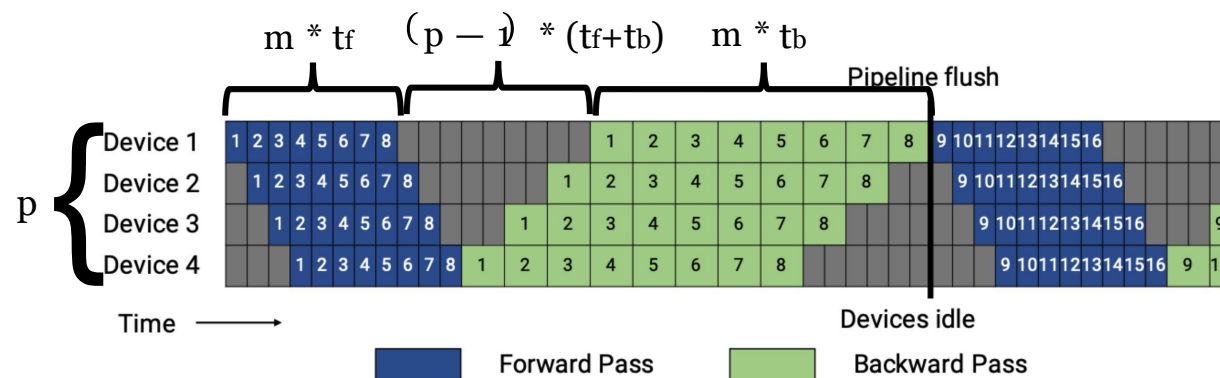
- m : micro-batches在一个mini-batch
- p :流水线阶段数
- 所有阶段都需要 t_f/ t_b 处理一个正向（反向） mini-batch



$$BubbleFraction = \frac{(p-1) * (t_f + t_b)}{m * t_f + m * t_b} = \frac{p-1}{m}$$

提高流水线并行效率

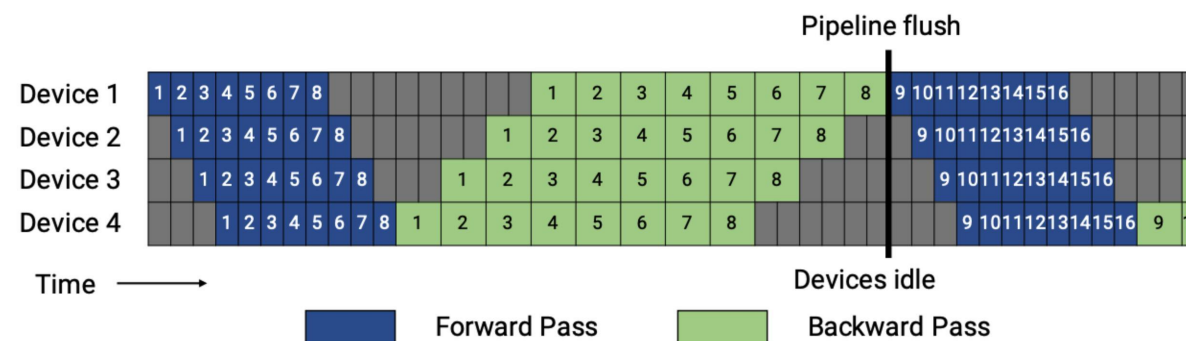
- m : number of micro-batches in a mini-batch
 - 增加mini-batch size或减少micro-batch size
 - 警告: mini-batch过大会导致精度损失; micro-batch减少 GPU 利用率
- p :流水线阶段数
 - 减少流水线深度
 - 注意: 增加阶段大小



$$BubbleFraction = \frac{(p - 1) * (t_f + t_b)}{m * t_f + m * t_b} = \frac{p - 1}{m}$$

模型并行性:流水线内存需求

- 一个问题:在反向繁殖之前,需要保留所有 mini-batch 的中间激活

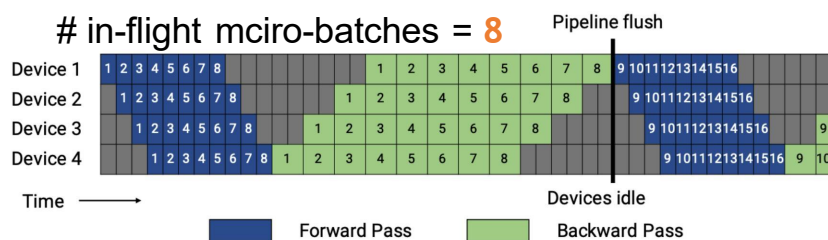


可以改进流水线计划以减少内存需求吗?

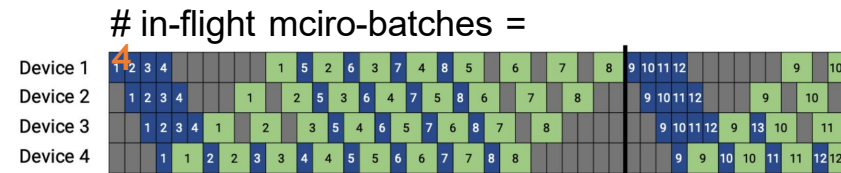
流水线并行性与 1F1B 时间表

- One-Forward-One-Backward在稳态中
- 限制正在执行中的 mini-batch 的数量到流水线深度
- 减少流水线并行的内存占用
- 不减少流水线气泡

能减少流水线气泡吗?



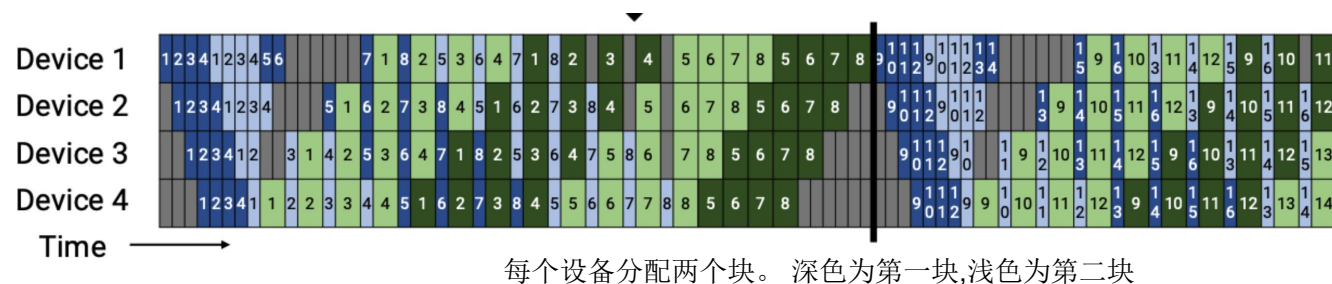
流水线并行与 GPipe 的时间表



流水线并行性与 1F1B 计划

1F1B 调度交错的流水线并行性

- 进一步将每个阶段细分为 v 个子阶段 $\frac{t_f}{v} (\frac{t_b}{v})$
- 每个子阶段的前进(后退)时间是



$$BubbleFraction = \frac{(p-1) * \frac{(t_f+t_b)}{v}}{m * t_f + m * t_b} = \frac{1}{v} * \frac{p-1}{m}$$

以增加沟通为代价减少气泡时间

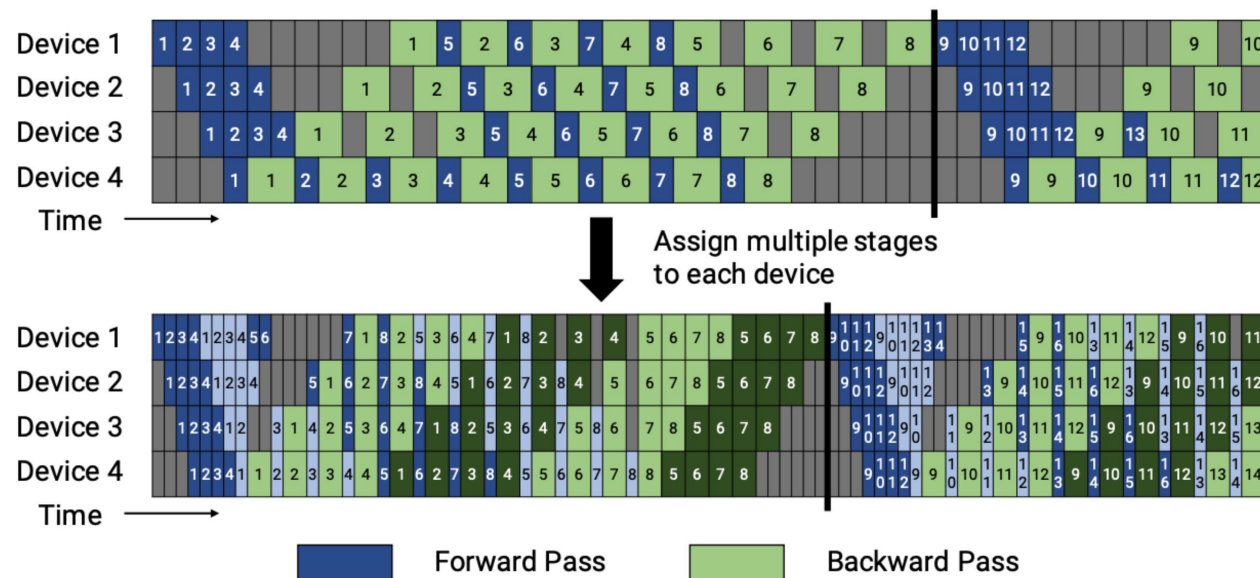
1F1B 调度交错的流水线并行性

流水线并行 with 1F1B Schedule

$$\text{BubbleFraction} = \frac{p-1}{m}$$

流水线并行 with interleaved 1F1B Schedule

$$\text{BubbleFraction} = \frac{1}{v} * \frac{p-1}{m}$$



流水线并行

- 现在，已经拥有了理解Llama 3.1中流水线并行性方法的所有要素，该方法使用1F1B设置，具有交错阶段，并在深度优先和广度优先之间可调的优先级设置：

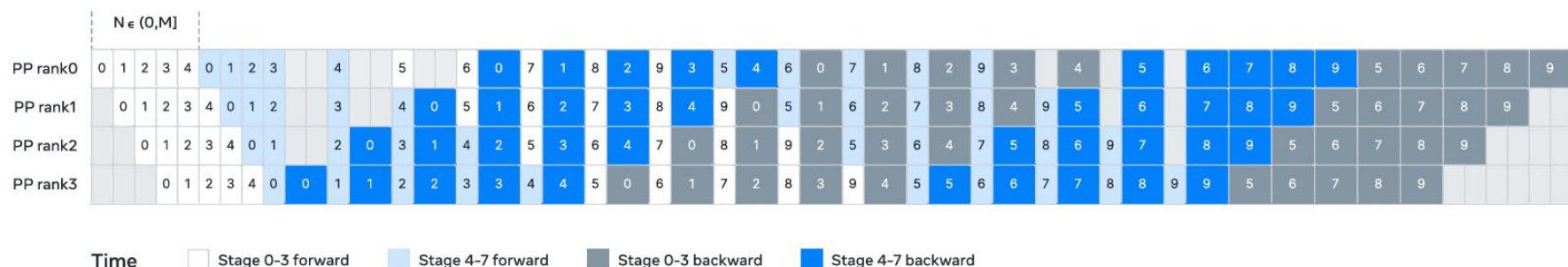


Figure 6 Illustration of pipeline parallelism in Llama 3. Pipeline parallelism partitions eight pipeline stages (0 to 7) across four pipeline ranks (PP ranks 0 to 3), where the GPUs with rank 0 run stages 0 and 4, the GPUs with P rank 1 run stages 1 and 5, etc. The colored blocks (0 to 9) represent a sequence of micro-batches, where M is the total number of micro-batches and N is the number of continuous micro-batches for the same stage's forward or backward. Our key insight is to make N tunable.

- 然而，还没有达到可能的流水线时间表的尽头，最近有人提出了一些方法，将气泡几乎降至零！例如，这些技术被用于DeepSeek-V3/R1的实现

流水线并行

零气泡和双流水线

最近提出了更多复杂的方法来减少气泡，这些方法接近“零气泡”状态，例如DeepSeek-V3/R1中的流水线实现方法，称为双流水线。这里的秘诀是在更细粒度的层面上分割涉及的操作，以便以最有效的方式交错它们。

在DeepSeek-V3技术报告中，作者指出他们的设置实现了“接近零的全对全通信开销。”

简要地看看这是如何工作的

- 通过总结Sea AI Lab的零气泡工作[10]，这是双流水线的前身。这里的观察基本是，通过矩阵乘法的反向传递实际上涉及两个独立的操作：输入的反向操作（B）和权重的反向操作（W）。

流水线并行

让简要地看看这是如何工作的

- 虽然B的反向传递，即输入的反向操作，对于执行下层反向传递是必要的，但权重W的反向传递则不是，通常只需要在优化器步骤之前执行。可以在以下图表（来自零气泡论文）中看到这一点：

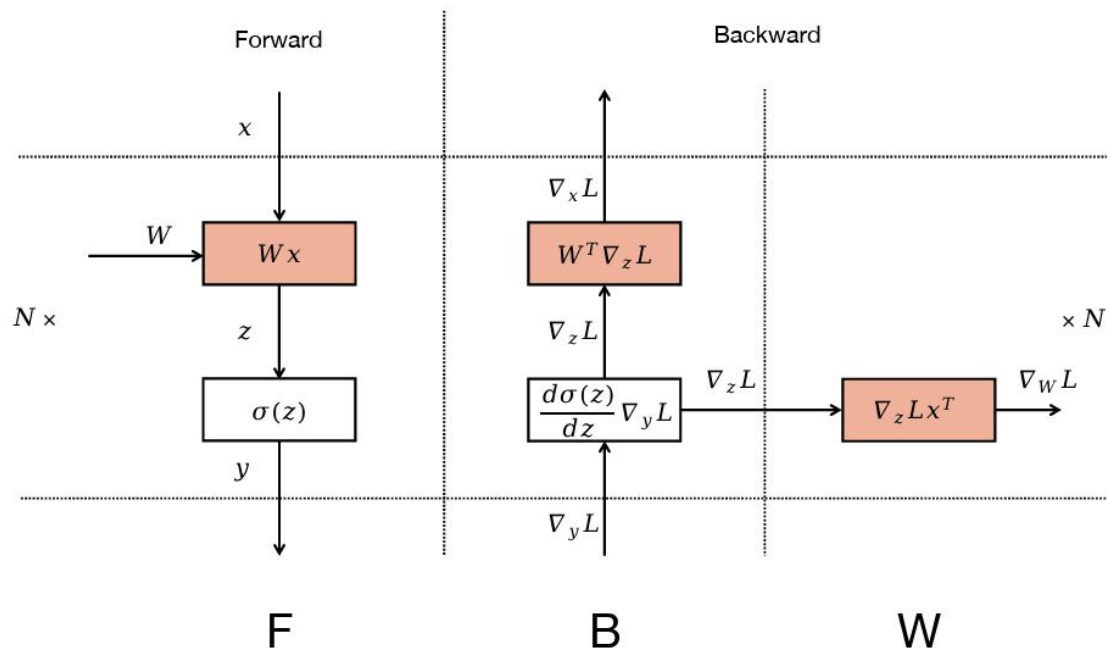


Figure 1: Computation Graph for MLP.

流水线并行

这意味着W可以在对应B之后灵活地安排在任何地方。

- 这允许战略性地放置W以填充流水线气泡。右上角的ZB-H2计划是一个利用这种细粒度分解的（理论上的）零气泡计划的例子。

在顶部（来自《Zero Bubble》论文的图2）：
经典的1F1B计划，交错正向和反向传递，但保留一个粗粒度的反向传递。

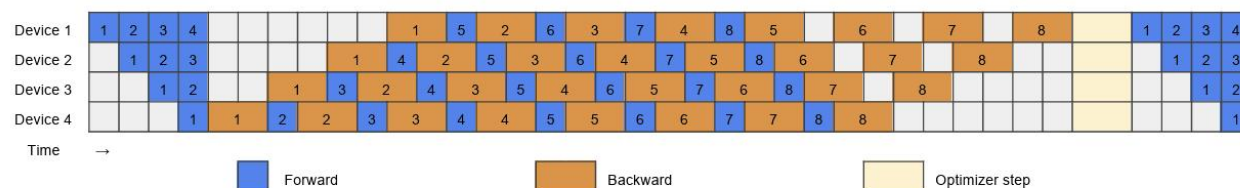


Figure 2: 1F1B pipeline schedule.

在底部（来自《Zero Bubble》论文的图3）：
两个手工制作的计划将反向传递分割成更细粒度的B和B以及W和W操作。下方的计划是一个（理论上的）零气泡计划，利用这种细粒度分解的优势。

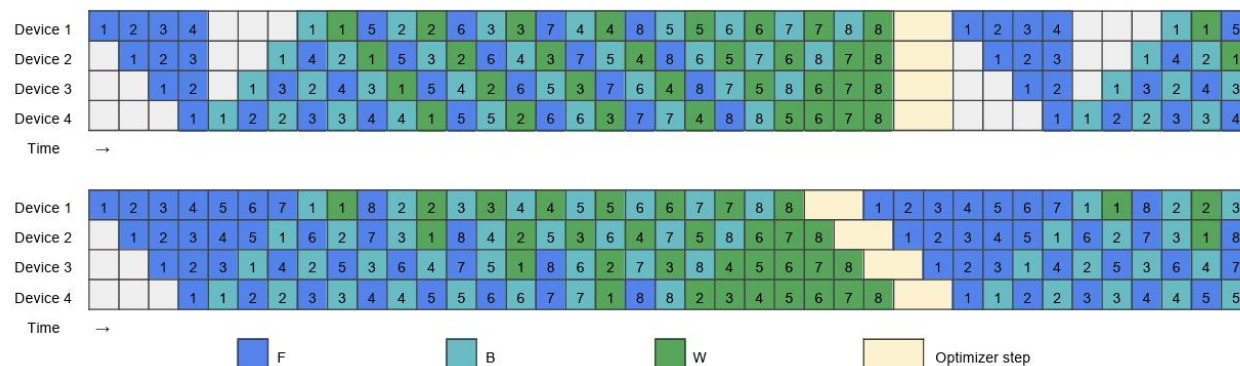


Figure 3: Handcrafted pipeline schedules, top: ZB-H1; bottom: ZB-H2

流水线并行

DeepSeek的DualPipe，随其V3技术报告一同推出

- 是这种分解方法在从PP维度两端传播的两个流的情况下的扩展，这些流被交错以进一步减少GPU中的空闲时间。此调度方案在以下调度图中显示——它比之前的更加复杂：

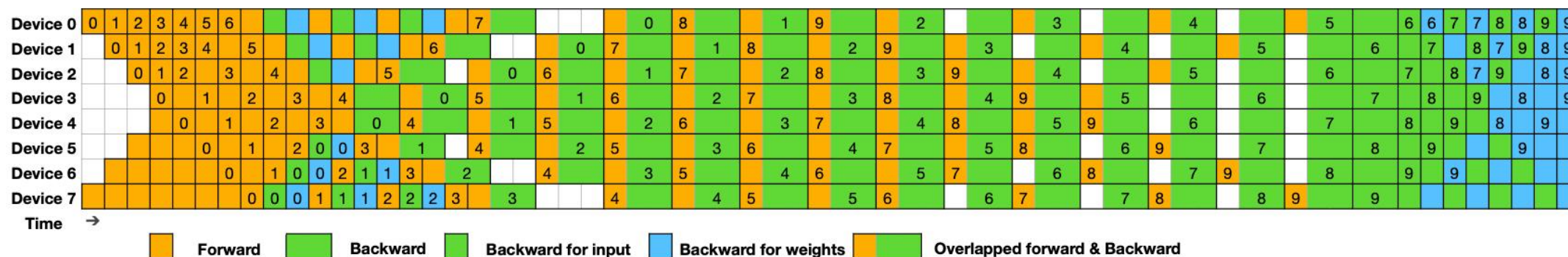


Figure 5 | Example DualPipe scheduling for 8 PP ranks and 20 micro-batches in two directions. The micro-batches in the reverse direction are symmetric to those in the forward direction, so we omit their batch ID for illustration simplicity. Two cells enclosed by a shared black border have mutually overlapped computation and communication.

流水线并行

一般来说

- 全面优化此类复杂调度需要仔细测量各种细粒度操作的持续时间，并通过解决整数线性规划（ILP）问题来最小化最终气泡时间。（例如，参见“零气泡”论文，其中讨论了用于执行此类调度的启发式算法。）
- 因此，零气泡和DualPipe调度过于复杂。

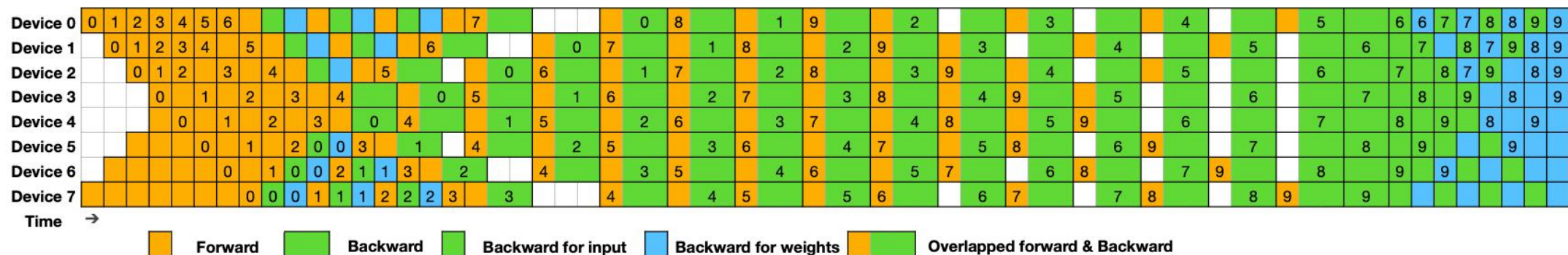


Figure 5 | Example DualPipe scheduling for 8 PP ranks and 20 micro-batches in two directions. The micro-batches in the reverse direction are symmetric to those in the forward direction, so we omit their batch ID for illustration simplicity. Two cells enclosed by a shared black border have mutually overlapped computation and communication.

目录

Contents

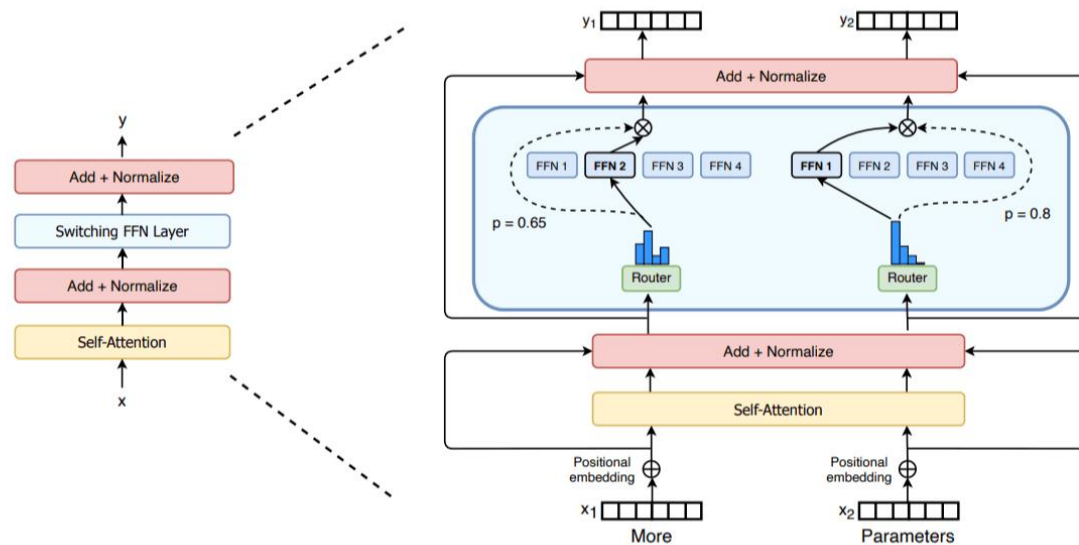
- 1 张量并行
- 2 上下文并行
- 3 流水线并行
- 4 专家并行**
- 5 并行对比与扩展

专家并行

专家混合范式

专家混合范式最近随着GPT-4、Mixtral、DeepSeek-V3/R1等模型而受到关注和重视。基本想法是，可以在每一层拥有多个并行模块，而不是只有一个前馈模块，并将标记通过它们进行不同处理。

来自《Switch Transformers》论文的
MoE层插图

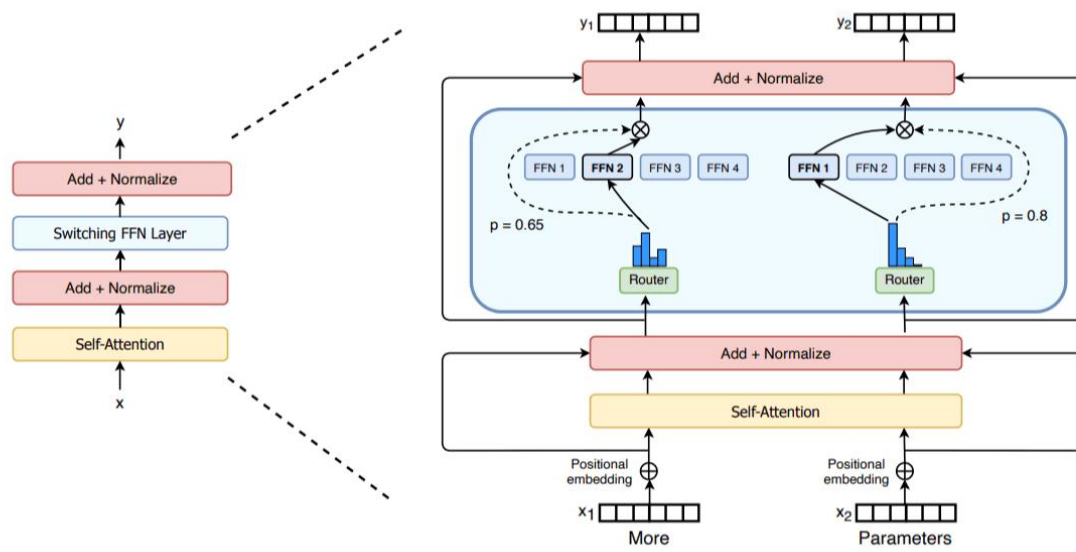


专家并行

专家混合范式

- 与稠密模型相比，预训练速度更快
- 与具有相同参数数量的模型相比，具有更快的推理速度
- 需要大量显存，因为所有专家系统都需要加载到内存中
- 在微调方面存在诸多挑战，但近期的研究表明，对混合专家模型进行指令调优具有很大的潜力。

来自《Switch Transformers》论文的
MoE层插图



专家并行

作为一种基于 Transformer 架构的模型，混合专家模型主要由两个关键部分组成：

- 稀疏 MoE 层: 这些层代替了传统 Transformer 模型中的前馈网络 (FFN) 层。MoE 层包含若干“专家”(例如 8 个)，每个专家本身是一个独立的神经网络。在实际应用中，这些专家通常是前馈网络 (FFN)，但它们也可以是更复杂的网络结构，甚至可以是 MoE 层本身，从而形成层级式的 MoE 结构。
- 门控网络或路由: 这个部分用于决定哪些令牌 (token) 被发送到哪个专家。例如，在下图中，“More”这个令牌可能被发送到第二个专家，而“Parameters”这个令牌被发送到第一个专家。有时，一个令牌甚至可以被发送到多个专家。令牌的路由方式是 MoE 使用中的一个关键点，因为路由器由学习的参数组成，并且与网络的其他部分一同进行预训练。

专家并行

专家并行

MoE层的设计使得在专家维度上实现并行化变得容易，称之为专家并行（EP）。由于前馈层是完全独立的，可以简单地将每个专家的前馈层放在不同的工作者上。与TP相比，这种方法要轻量得多，因为不需要分割矩阵乘法；只需要将一个标记的隐藏状态路由到正确的专家。

在实践中，EP通常与其他形式的并行化结合使用，例如数据并行化

- 这是因为EP只影响MoE层，不会分割输入标记（与上下文并行化不同，上下文并行化会沿着序列长度维度分割标记）。
- 这意味着如果只使用EP，的GPU将对所有非MoE块进行冗余计算。通过将EP与DP结合，可以在的GPU上有效地分割专家和输入批次

专家并行

在实践中，EP通常与其他形式的并行化结合使用，例如数据并行化

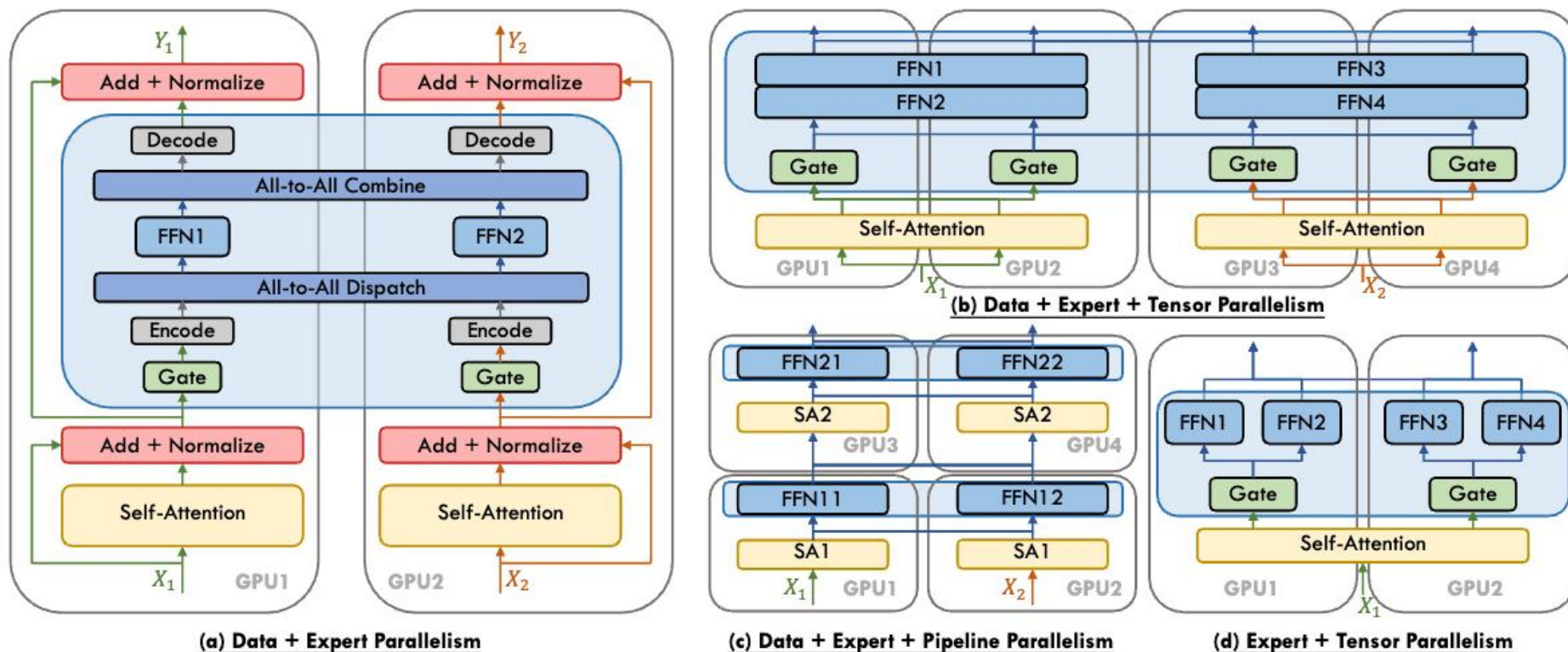


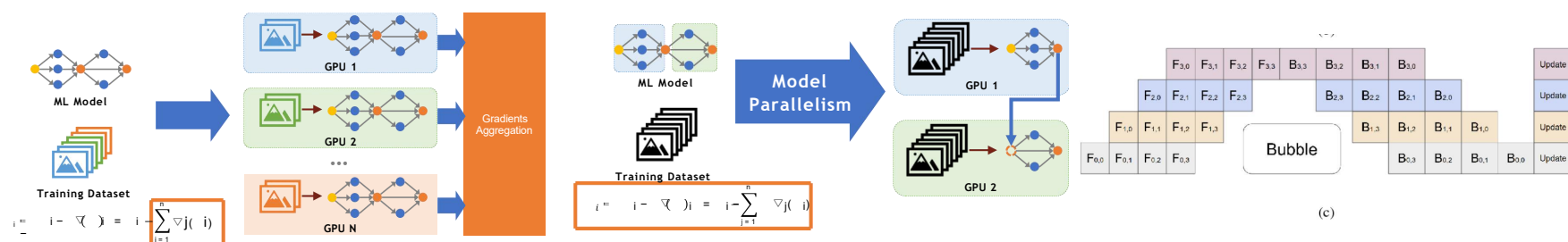
Fig. 8. Schematic depiction of diverse parallel strategies for MoE. For clarity and conciseness, this illustration omits some All-to-All, All-Reduce, Point-to-Point communication within parallelism, and Normalization, Encode, Decode, Gate in subfigures (b), (c), and (d).

目录

Contents

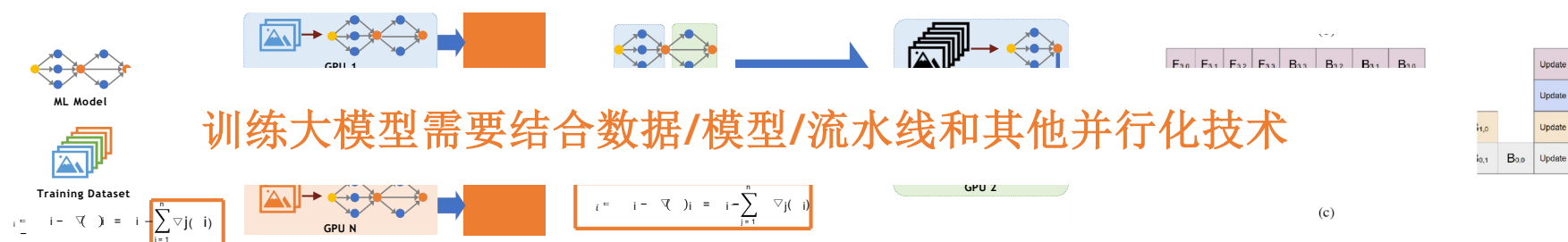
- 1 张量并行
- 2 上下文并行
- 3 流水线并行
- 4 专家并行
- 5 并行对比与扩展**

比较数据/张量模型/流水线模型并行性



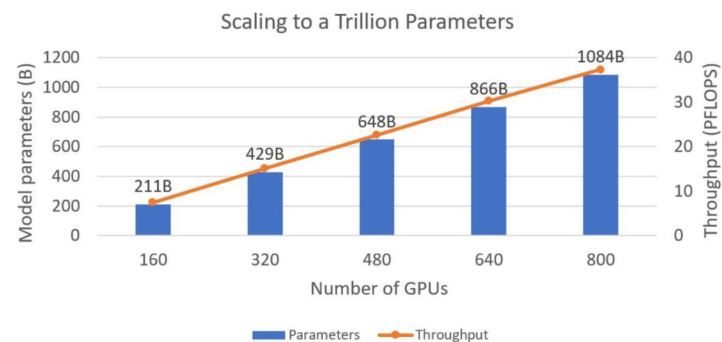
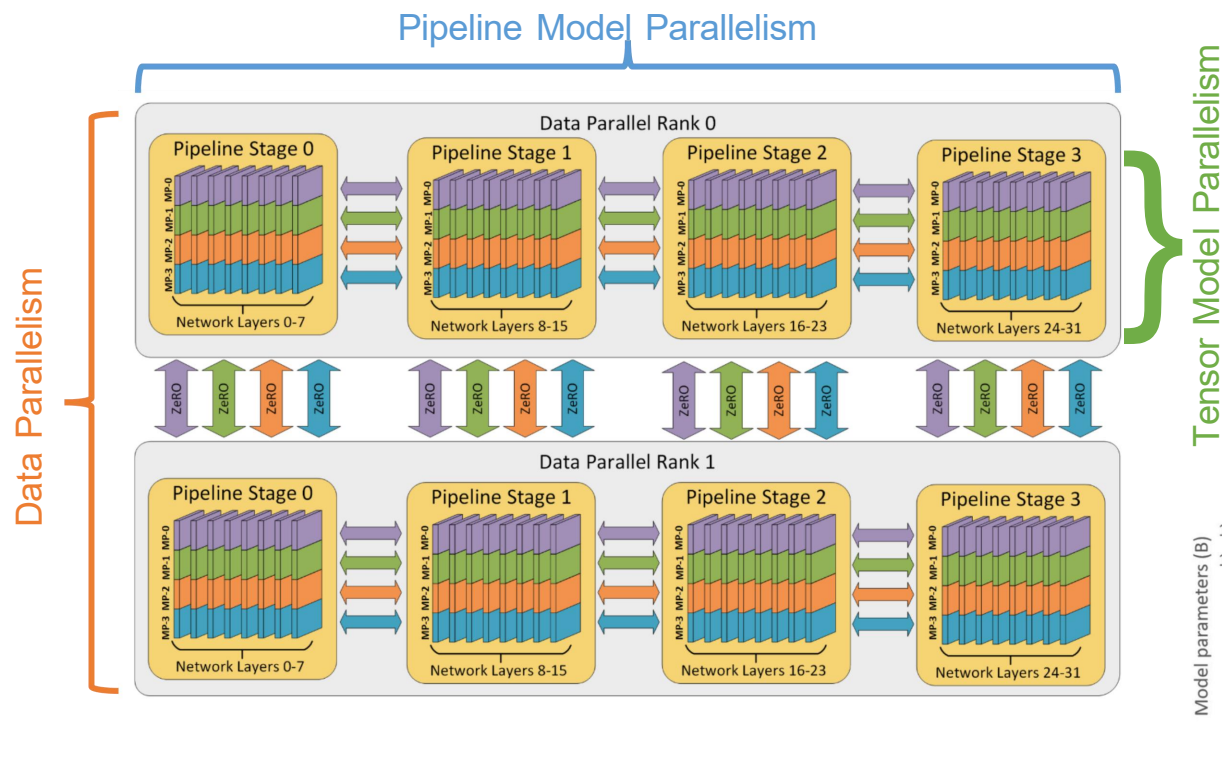
	数据并行	张量模型并行	流水线模型并行性
Pros	✓ 大规模并行化 ✓ 前进/后退时无需通信	✓ 支持训练大模型 ✓ 对于参数众多的模型有效	✓ 支持大批量培训 ✓ 对于深度模型有效
Cons	❖ 不适用于无法容纳 GPU 的模型 ❖ 对于参数众多的模型,不要缩放	❖ 并行性有限; 不能扩展到大量 GPU ❖ 需要向前/向后传输中间结果	❖ 有限利用:前后气泡

比较数据/张量模型/流水线模型并行性



	数据并行	张量模型并行	流水线模型并行性
Pros	✓ 大规模并行化 ✓ 前进/后退时无需通信	✓ 支持训练大模型 ✓ 对于参数众多的模型有效	✓ 支持大批量培训 ✓ 对于深度模型有效
Cons	❖ 不适用于无法容纳 GPU 的模型 ❖ 对于参数众多的模型,不要缩放	❖ 并行性有限; 不能扩展到大量 GPU ❖ 需要向前/向后传输中间结果	❖ 有限利用:前后气泡

示例:DeepSpeed 中的 3D 并行性



5D并行

5D并行

- ✓ 数据并行 (DP) - 沿着批量维度
- ✓ 张量并行 (TP) - 沿着隐藏维度
- ✓ 序列和上下文并行 (SP/CP) - 沿着序列维度
- ✓ 流水线并行 (PP) - 沿着模型层
- ✓ 专家并行 (EP) - 沿着模型专家

三种可以与数据并行结合以减少内存使用的ZeRO策略

- ZeRO-1 – 在DP副本之间分片优化器状态
- ZeRO-2 – 在DP副本之间分片优化器状态和梯度
- ZeRO-3 – 在DP副本之间分片优化器状态、梯度和参数

5D并行

流水线并行和ZeRO-3

- ✓ 流水线并行和ZeRO-3都是将模型权重分片到多个GPU上，并在模型深度轴上进行通信/计算（例如，在ZeRO-3中，在计算的同时预取下一层）。
- ✓ 这意味着在这两种情况下，每个设备都计算了完整的层操作，这与TP或EP不同，例如，计算是在子层单元上进行的。

然而，PP和ZeRO-3方法之间有几个主要区别：

5D并行

然而，PP和ZeRO-3方法之间有几个主要区别：

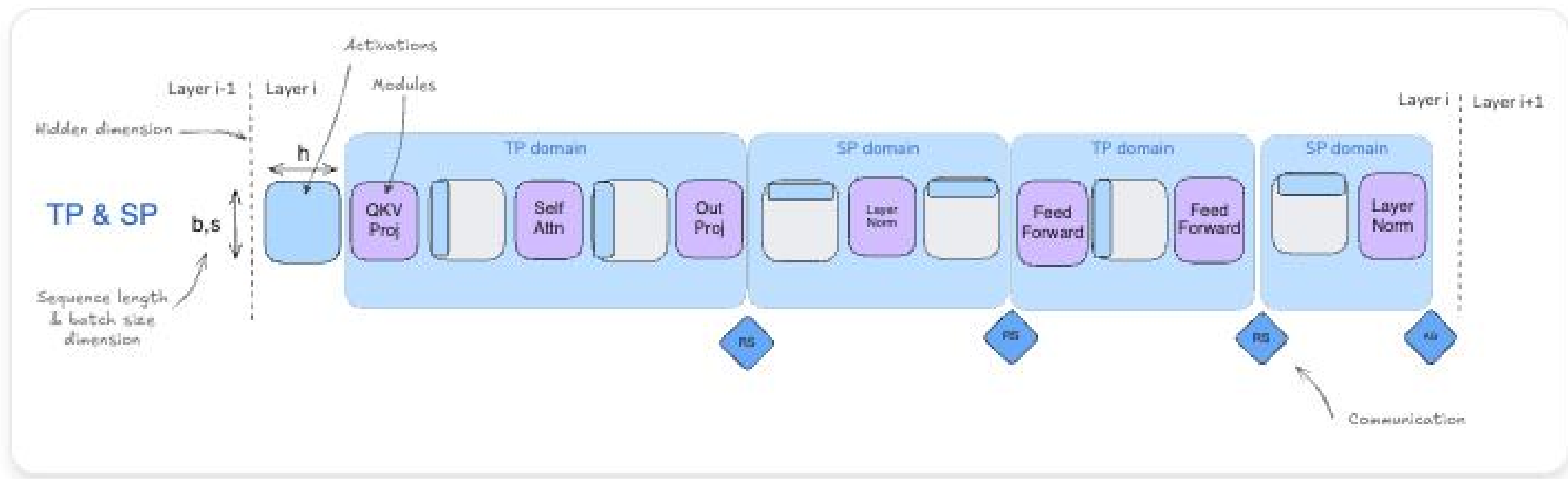
	ZeRO-3	流水线并行
每个计算单元存储...	仅为一层中的一小部分	一整层
通信用于传输...	权重	激活
编排	模型无关	模型无关
实施挑战	处理模型分区和通信的复杂性	处理高效PP时间表的复杂性
缩放考虑因素	偏好大 mbs 和 seq_len 以隐藏通讯	偏好大 $grad_acc$ 来隐藏气泡

ZeRO-3和PP解决相同的挑战，但采用不同的方法，它们之间的选择将取决于是否决定将通信重点放在权重或激活的传输上。虽然它们可以结合使用，但在实践中很少这样做，因为这需要显著增加全局批量大小以分摊通信成本，从而在全局批量大小、模型大小、网络带宽和训练效率之间产生权衡。如果决定将它们结合起来，ZeRO-3应配置为在PP微批次的系列中保持权重在内存中，以尽可能减少不必要的通信开销。

5D并行

另一方面，ZeRO-1和ZeRO-2专注于优化器状态和梯度，可以轻松与流水线并行结合

- ✓ 这些组合不会引发任何特别的新挑战。例如，DeepSeek-v3的训练使用了结合了PP和ZeRO-1。
- ✓ 张量并行（包括序列并行）与流水线并行和ZeRO-3自然互补，可以与它们结合，因为它依赖于矩阵乘法的分配性质，这使得权重和激活可以在组合之前被分割并独立计算。



5D并行

不希望仅使用TP来实现并行化的主要原因在于，在实践中，TP有两个限制

- ✓ 首先，由于其通信操作是计算关键路径的一部分，因此很难在超过一定点后进行良好的扩展，之后通信开销开始占据主导地位。
- ✓ 其次，与ZeRO和PP不同，它们是模型无关的，TP需要仔细处理激活分片——有时是在TP区域中的隐藏维度上，有时是在SP区域中的序列维度上——这使得正确实现变得更加繁琐，并需要模型特定的知识来确保在整个模型中实现适当的分片模式。

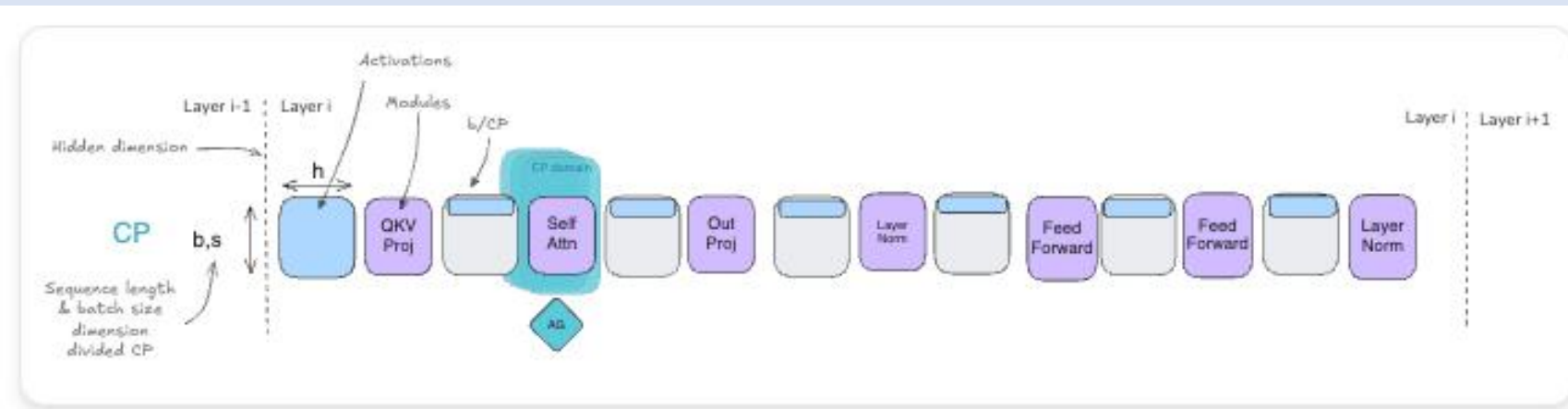
因此，在结合并行化策略时，TP通常会被保留用于高速节点内通信

- 而ZeRO-3或PP可以用于跨越低速节点间通信的并行化组，因为它们的通信模式需要的带宽更少（对于PP）或可以更容易地与计算重叠（对于ZeRO-3）。
- 结合这些技术时，主要考虑的是为每个并行化维度有效地组织GPU组，以最大化吞吐量并最小化通信开销，同时注意TP的扩展限制。例如，用于TP通信的GPU组应保留在节点内部。

5D并行

上下文并行和专家并行也有助于划分激活，可以看作是TP的补充

- ✓ CP处理长序列，而EP使分布式混合专家训练成为可能，它们可以结合使用而不会出现任何特定问题。
- ✓ CP特别针对使用非常长的序列进行训练的挑战，通过在GPU上沿序列维度划分激活。虽然大多数模块，如MLP和LayerNorm，可以独立处理这些划分的序列，但注意力块需要通信，因为每个标记都需要访问整个序列的键/值。
- ✓ 正如在CP部分所看到的，这通过重叠计算和通信的环状注意力模式得到有效处理。CP在扩展到极端序列长度（128k+标记）时尤其有价值，即使使用完整的激活重新计算，注意力在单个GPU上的内存需求也会过高。



5D并行

专家并行专门针对训练MoE模型时面临的挑战

- ✓ 通过在GPU间划分专门的“专家”并动态路由标记到相关的专家在计算期间。EP中的关键通信操作是将标记路由到其分配的专家并收集结果。
- ✓ 虽然这引入了一些通信开销，但它能够显著扩展模型容量，因为每个标记仅在推理（和训练）期间由总参数中更小的一部分处理。在分布式训练/推理方面，当模型扩展到大量专家时，在GPU间划分专家变得相关。

